UNIVERSITY OF MAURITIUS
Knowledge for a Better World

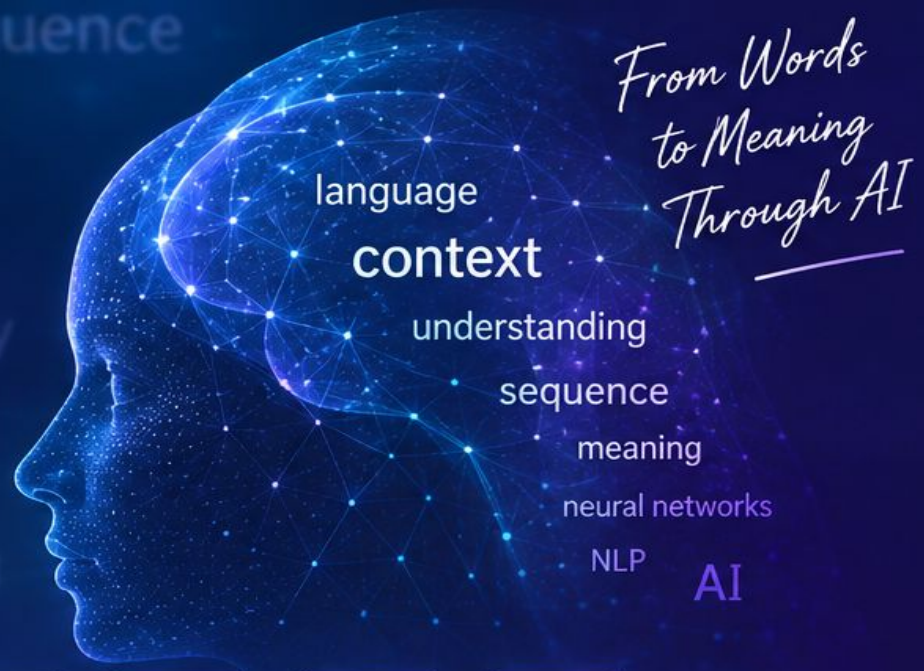
SIS 3204Y(5)
NATURAL LANGUAGE PROCESSING

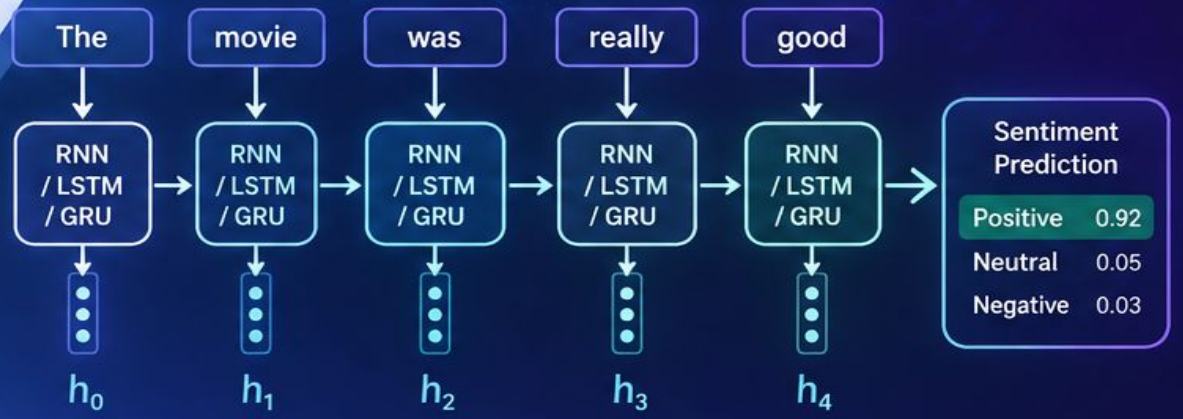
WEEK 5

Recurrent Neural Networks for NLP

RNN, LSTM AND GRU

Learning from Sequences, Understanding Context





SEQUENCES AND CONTEXT | RNN, LSTM AND GRU | REAL-WORLD NLP APPLICATIONS | TOWARDS MODERN NLP

Build Understand Apply

UNIVERSITY OF MAURITIUS
Knowledge for a Better World

SIS 3204Y(5)
NATURAL LANGUAGE PROCESSING

WEEK 5
Recurrent Neural Networks for NLP
RNN, LSTM and GRU

LEARNING OUTCOMES

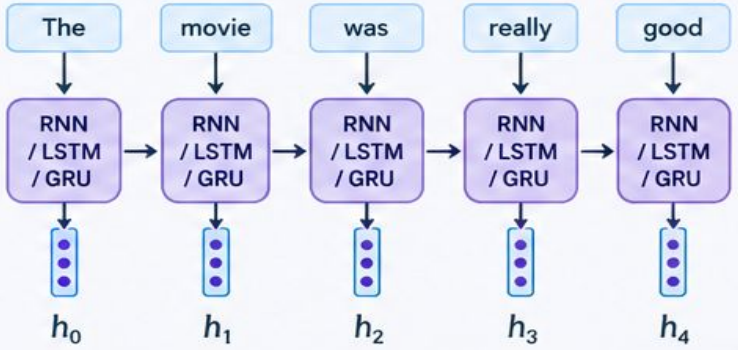
By the end of Week 5, students should be able to:

1. **Explain** why sequence and word order matter in natural language processing.
2. **Describe** the limitations of standard (feed-forward) neural networks for sequential text.
3. **Explain** the architecture and operation of a recurrent neural network (RNN).
4. **Illustrate** how an RNN processes a text sequence using hidden states.
5. **Explain** the vanishing gradient problem and its impact on learning long-term dependencies.
6. **Describe** how Long Short-Term Memory (LSTM) networks address the limitations of basic RNNs, including the role of the forget, input and output gates.
7. **Explain** the architecture and operation of a Gated Recurrent Unit (GRU), including the reset and update gates.
8. **Compare** RNN, LSTM and GRU in terms of architecture, strengths, limitations and suitable use cases.
9. **Apply** RNN, LSTM or GRU models to solve simple NLP tasks such as text classification or sequence labelling.
10. **Critically discuss** the strengths and limitations of recurrent neural networks for NLP, and motivate the need for attention-based models (Transformers).

Sequences Unlock Meaning

"Dog bites man" ≠ "Man bites dog"

The same words, different meaning.



"Understanding sequences today, for smarter language technologies tomorrow."

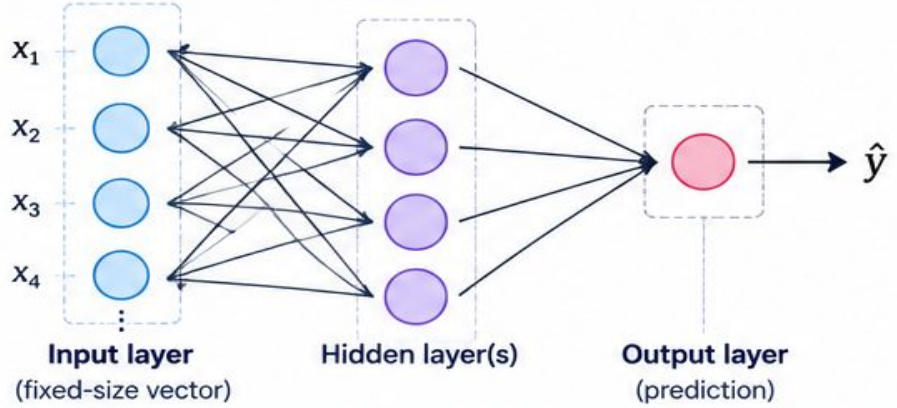
LANGUAGE
PEOPLE
AI
OPPORTUNITIES

From Feed-Forward Neural Networks to Sequence Models

Standard (feed-forward) neural networks process fixed-size inputs and do not maintain memory of previous inputs.
To work with language, we need models that can handle sequences and capture word order and context.

Feed-Forward Neural Network (FNN)

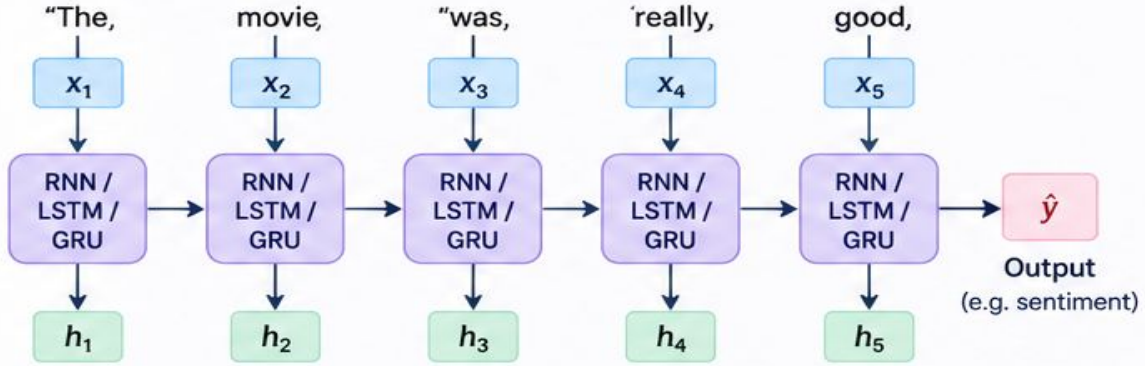
Processes a fixed-size input and produces an output.



- Takes a fixed-length input vector (e.g. a flattened image, a bag of words representation).
- All inputs are processed at once, independently.
- No memory of previous inputs.
- Suitable for non-sequential data.

Sequence Models (e.g. RNN, LSTM, GRU)

Process a sequence of inputs, maintaining a hidden state (memory).



- Takes a sequence of inputs (e.g. a sequence of word embeddings).
- Maintains a hidden state that is updated at each time step.
- Can capture word order and context.
- Suitable for sequential data such as text, speech or time series.

Why do we need sequence models?

- The meaning of a sentence depends on the order of the words.
- A feed-forward network treats inputs independently and cannot model the relationships between words in a sequence.
- Sequence models (RNN, LSTM, GRU) are designed to process text step by step and retain information from previous words.

Example: Word Order Matters

"Dog bites man"	→	Meaning: The dog is the subject.
"Man bites dog"	→	Meaning: The man is the subject.

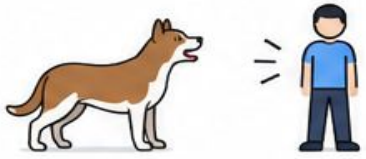
Same words, differentl order, different meaning!

Why Word Order Matters in NLP

In natural language, the order of words changes the meaning of a sentence.
A model must be able to capture this sequential information to understand and generate language correctly.

Same words, different meaning

Dog **bites** **man**



Meaning:
The dog is the subject (the doer).

Man **bites** **dog**



Meaning:
The man is the subject (the doer).

Other examples

I love my dog.	Expresses affection.
My dog loves I.	Grammatically incorrect (and has a different meaning).
She is not happy.	Negative meaning.
She is happy not.	Different meaning (awkward/incorrect).
They will book the flight tomorrow.	Future action.
Tomorrow they will book the flight.	Same meaning, different emphasis.
I saw the man with a telescope.	I used a telescope to see the man.
I saw the man with the telescope.	The man had a telescope.

Why it matters in NLP

1. **Meaning depends on sequence**
   The meaning of a sentence is determined by the order of its words.
2. **Grammar and syntax**
   Word order defines grammatical structure and relationships between words.
3. **Context and semantics**
   Changing the order can completely change the interpretation.
4. **NLP tasks rely on word order**
   Tasks such as machine translation, sentiment analysis, question answering and text generation require understanding of word order.
5. **Models must capture sequential information**
   Standard (feed-forward) neural networks treat inputs independently and do not model word order. Sequence models (e.g. RNN, LSTM, GRU) are designed to handle this.

Key Takeaway: In NLP, the same set of words can express very different meanings depending on their order. Therefore, we need models that can process text as a sequence and capture word order and context.



Text as a Sequence

In NLP, a text is viewed as an ordered sequence of tokens (words, subwords or characters).
Each token has a position in the sequence, and the order carries important information about meaning.

A sentence as a sequence of tokens

"The movie was really good."

Tokens (words)	The	movie	was	really	good	.
	1	2	3	4	5	6

Different types of sequences

Word sequence	The	movie	was	really	good	.				
Subword sequence (e.g. BPE)	The	mov	##ie	was	really	good	.			
Character sequence	T	h	e	_	m	o	v	i	e	...

From tokens to model inputs

1 Tokenisation
Split the text into tokens (words, subwords or characters).

2 Convert to numerical representation
Each token is mapped to an index (or directly to an embedding).

	The	movie	was	really	good	.
Token indices (example)	12	235	18	401	620	9

3 Convert to embeddings (vectors)
Each token is represented as a dense vector.

	The	movie	was	really	good	.
Embedding vectors (e.g. 300-dim)	↓	↓	↓	↓	↓	↓

Key Points

1. **Text is sequential**
   A text is a sequence of tokens in a specific order.
2. **Order matters**
   The same tokens in a different order can have a different meaning.
3. **Tokens can be words, subwords or characters**
   The choice of tokenisation method depends on the task.
4. **Each token is converted to a numerical representation**
   Usually an index, which is then mapped to a dense embedding vector.
5. **The sequence is fed to a model**
   Sequence models (e.g. RNN, LSTM, GRU) process the tokens one by one (or in parallel, as in Transformers) while taking the order into account.

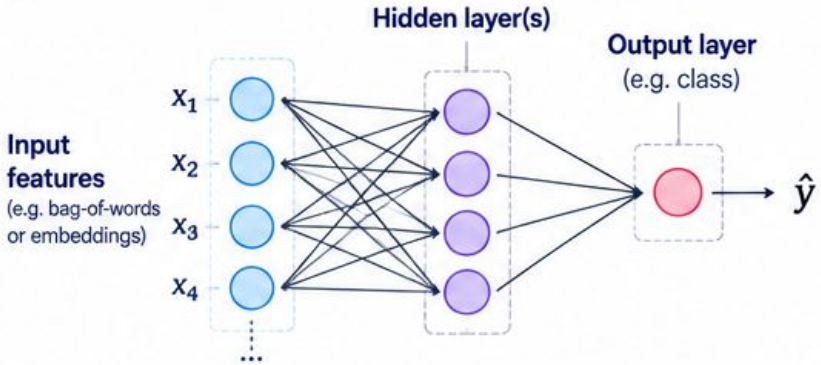
Key Takeaway: In NLP, text is not just a collection of words — it is an ordered sequence of tokens.
Models must be able to capture and use this sequential information to understand meaning.

Why Standard Neural Networks Are Not Enough

Standard (feed-forward) neural networks treat all inputs independently and do not maintain any memory of previous inputs. This makes them unable to model the sequential nature of language, where the meaning of a word depends on the words around it.

Standard Neural Network (Feed-Forward)

Processes a fixed-size input without memory of previous words.



- Takes a fixed-size input vector (e.g. averaged word embeddings).
- Processes all inputs at once, independently.
- No notion of word order or sequence.
- Cannot remember previous words.
- Suitable for tasks where order does not matter (e.g. simple document classification with bag-of-words).

The Problem with Language

The meaning of a word depends on its context and position.

The cat sat on the mat.	Meaning: A cat is sitting.
The mat sat on the cat.	Meaning: Different meaning (usually nonsensical).
I saw the man with a telescope.	Meaning: I used a telescope.
I saw the man with the telescope.	Meaning: The man had a telescope.
She is not happy.	Meaning: Negative sentiment.
She is happy.	Meaning: Positive sentiment.

Key Limitations of Standard NNs for NLP

1. **No memory of previous words**
   Each word is treated independently, so the model cannot use earlier words to understand later words.
2. **Ignores word order**
   A bag-of-words or averaged embedding representation loses the sequence information.
3. **Cannot model long-range dependencies**
   Relationships between words that are far apart in a sentence cannot be captured.
4. **Limited contextual understanding**
   The same word may have different meanings in different contexts, which a standard NN cannot distinguish.
5. **Not suitable for sequential generation**
   Standard NNs cannot be used directly for tasks such as language modelling or text generation.

Key Takeaway:
Language is sequential. To understand and generate text, we need models that can use the order of words and maintain information from previous words. This motivates the use of recurrent neural networks (RNNs), LSTMs and GRUs.

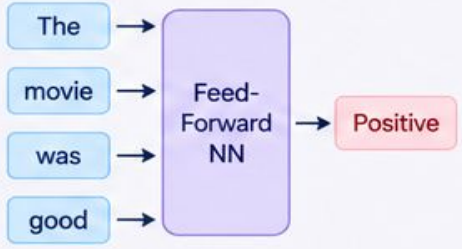
Introducing Memory into Neural Networks

To handle sequential data such as text, we need neural networks that can remember information from previous inputs. Recurrent neural networks (RNNs) introduce a form of memory through a hidden state, allowing the model to use past context.

From Feed-Forward to Recurrent Networks

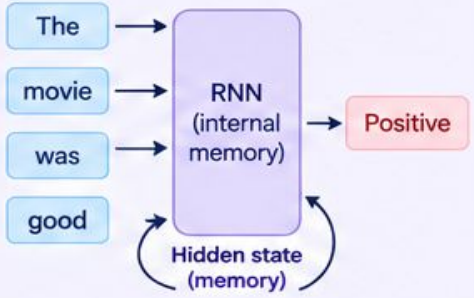
Feed-forward neural networks have no memory, whereas recurrent neural networks maintain al hidden state that captures information from previous words.

Feed-Forward Neural Network (no memory)



- Processes each input independently
- No memory of previous words
- Word order is not explicitly modelled

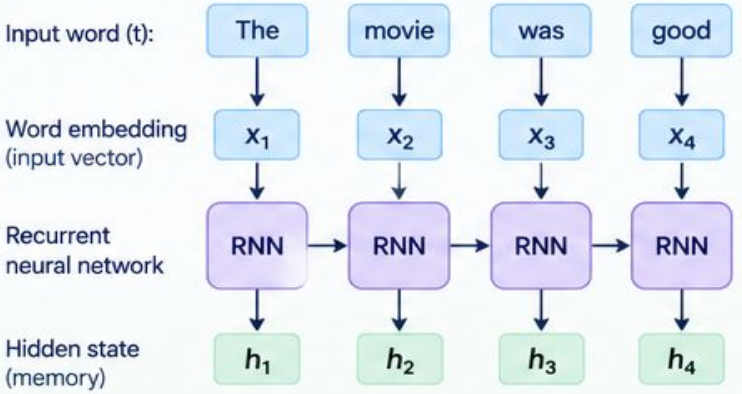
Recurrent Neural Network (with memory)



- Maintains a hidden state
- Uses information from previous words
- Can capture word order and context

The Hidden State as Memory

At each time step, the network updates a hidden state that summarises the information seen so far.



- The hidden state (h_t) acts as a memory of the previous words.
- It is updated at each time step using the current input and the previous hidden state.
- The final hidden state can be used for tasks such as text classification.

How Memory Helps

1. **Captures context**
   The model can use information from earlier words to understand the meaning of the current word.
2. **Handles variable-length input**
   Can process sentences of different lengths, one word at a time.
3. **Models dependencies**
   Can learn both short-term and longer-term relationships between words.
4. **Forms the basis for advanced models**
   RNNs, LSTMs and GRUs are all recurrent architectures that introduce different ways of managing memory.

Key Takeaway: By introducing a hidden state, recurrent neural networks give neural networks a form of memory, allowing them to process text as a sequence and capture the meaning that comes from word order and context.

What is a Recurrent Neural Network (RNN)?

A recurrent neural network (RNN) is a type of neural network designed to process sequential data by maintaining a hidden state (memory) that captures information from previous inputs.

Definition

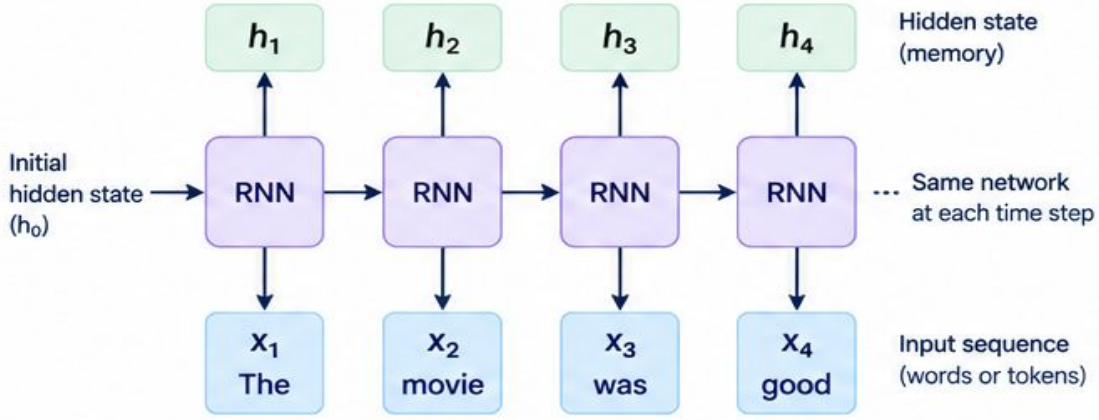
An RNN is a neural network that processes a **sequence of inputs one element at a time**, using a **hidden state** to remember information from previous inputs.

Key idea

Unlike standard (feed-forward) neural networks, an RNN has **memory.** The hidden state allows the model to use information from earlier words when processing the current word.

RNN at a high level

The same network is applied at each time step, with the hidden state carrying information from the past.



Key points

1. **Processes sequential data**
   Handles inputs in a specific order (e.g. words in a sentence).
2. **Maintains a hidden state**
   The hidden state captures information from previous inputs.
3. **Same parameters at each time step**
   The same network (weights) is reused for all positions in the sequence.
4. **Captures word order and context**
   Uses both the current input and the previous hidden state to produce the next hidden state.
5. **Widely used in NLP**
   Forms the basis for more advanced models such as LSTM and GRU.

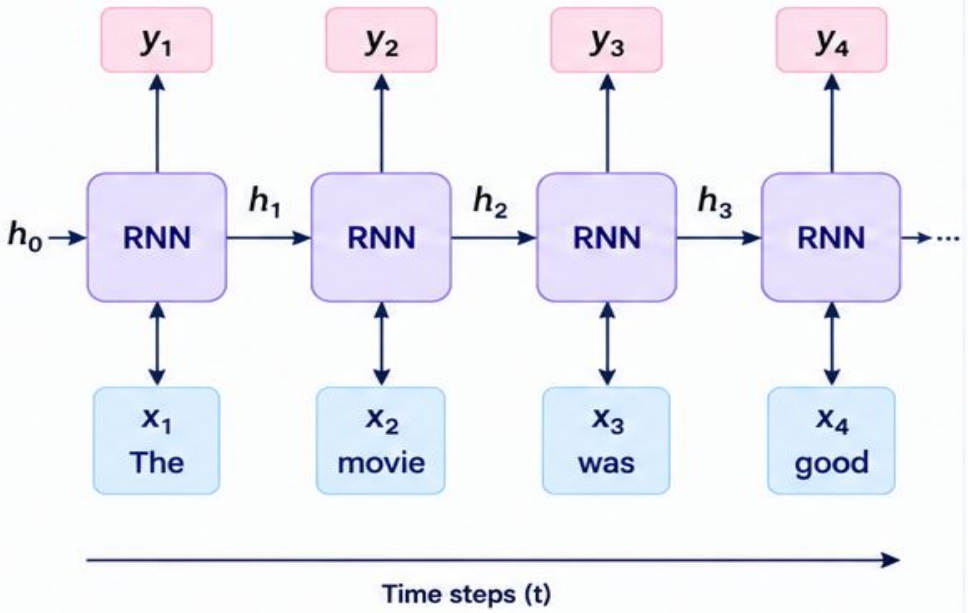
Key Takeaway:
RNNs introduce a form of memory into neural networks, allowing them to process text as a sequence and capture information from previous words.

The Basic RNN Architecture

An RNN processes a sequence one element at a time, maintaining a hidden state that is updated at each time step.
The same set of parameters is used for all time steps.

Unfolded RNN over time

The same RNN cell is applied at each time step.



RNN computations at time step *t*

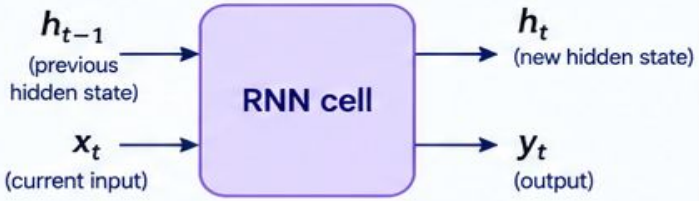
At each time step, the RNN takes the current input x_t and the previous hidden state h_{t-1} to compute the new hidden state h_t and an output y_t.

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b_h\right)$$

$$y_t = W_{hy}h_t + b_y$$

x_t	Input vector at time step t (e.g. word embedding)
h_t	Hidden state at time step t (captures information from previous inputs)
y_t	Output vector at time step t (e.g. class scores or next word distribution)
W_{xh}	Input-to-hidden weight matrix
W_{hh}	Hidden-to-hidden weight matrix (recurrent weights)
W_{hy}	Hidden-to-output weight matrix
b_h, b_y	Bias vectors
tanh	Activation function (introduces non-linearity)

Single RNN cell (compact view)



Key points

1. **Sequential processing**
   Processes the input one element at a time in a specific order.
2. **Parameter sharing**
   The same weights are used at every time step.
3. **Hidden state**
   Acts as a memory of previous inputs.
4. **Outputs at each time step**
   Can produce an output at each step (many-to-many) or only at the last step (many-to-one).

Key Takeaway: An RNN has a simple architecture: it takes the current input and the previous hidden state, updates the hidden state using a non-linear transformation, and produces an output. The same cell and parameters are used at all time steps.

Understanding the Hidden State

The hidden state is the memory of an RNN. It stores a compact representation of all the information seen so far and is updated at each time step using the current input and the previous hidden state.

What is the hidden state?

The hidden state ($\mathbf{h}_t$) is a vector that **summarises the information from all previous inputs** up to time step t.

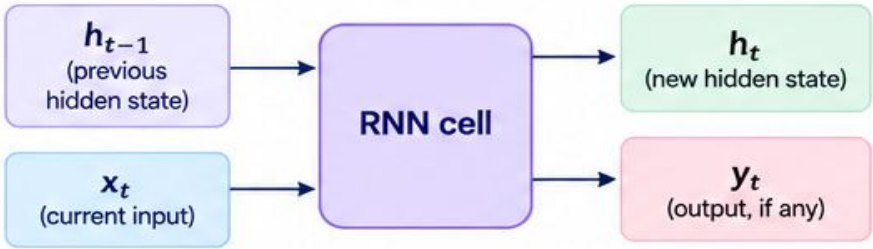
It allows the RNN to **maintain context** as it processes a sequence one element at a time.

Intuition

You can think of the hidden state as a **"summary"** of what the model has read so far. It is not the entire past, but a **condensed representation** that helps the model make decisions about what comes next.

Hidden state in the RNN computation

At each time step t, the hidden state is updated using the current input x_t and the previous hidden state h_{t-1}.



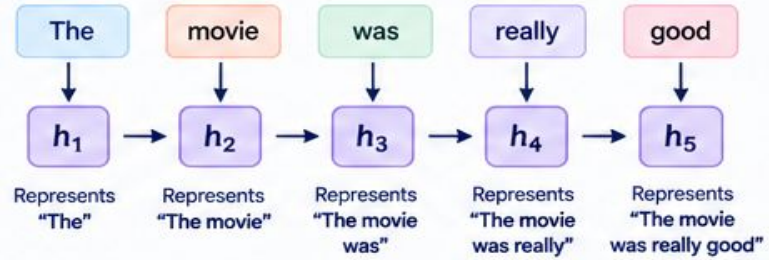
Update equation (simple RNN)

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b_h\right)$$

x_t	Input vector at time step t (e.g. word embedding)
h_{t-1}	Previous hidden state
h_t	Updated hidden state
W_{xh}	Input-to-hidden weight matrix
W_{hh}	Hidden-to-hidden weight matrix (recurrent weights)
b_h	Bias vector
tanh	Activation function (introduces non-linearity)

How the hidden state evolves (example)

The hidden state changes as each word is processed, capturing more context over time.



Key points

1. **The hidden state is a memory**
   It captures information from previous inputs.
2. **It is updated at each time step**
   Combines the current input and the previous hidden state.
3. **It has a fixed size**
   Acts as a compressed summary of the past, not the full history.
4. **It helps the model use context**
   Enables the RNN to make predictions based on earlier words.
5. **The same hidden state is used for different tasks**
   Its final value (or all hidden states) can be used for classification, sequence labelling or generation.

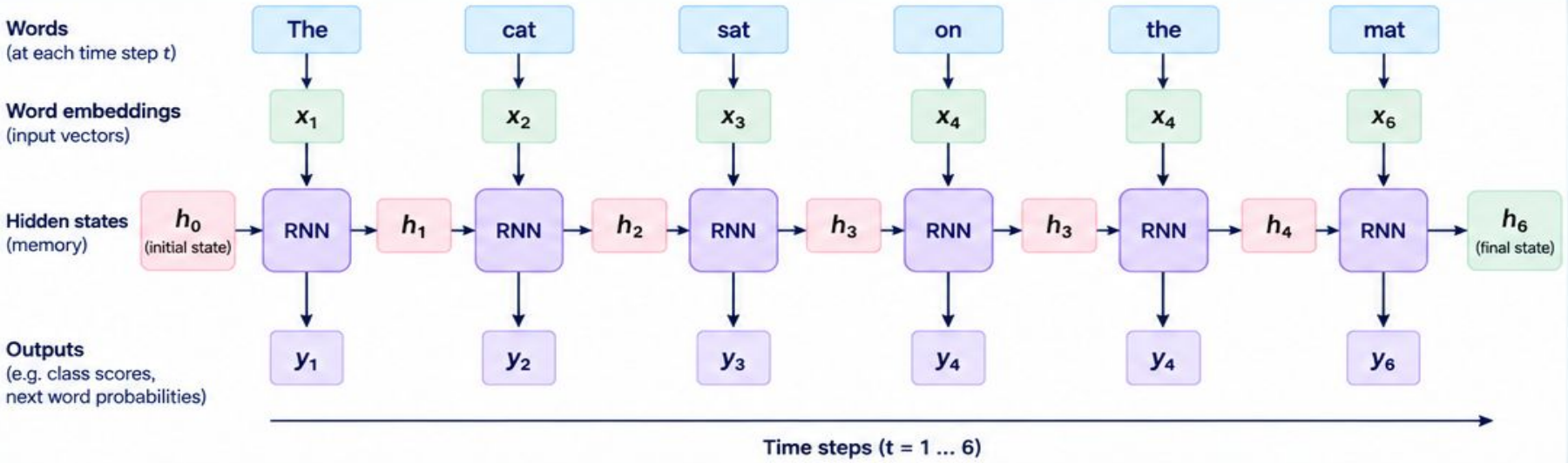
Key Takeaway: The hidden state is the RNN's memory — a compact representation of all the information seen so far. By updating it at each time step, the RNN can use context to better understand and process a sequence.

How an RNN Processes a Sentence

An RNN reads a sentence one word at a time. At each time step, it takes the current word (as a vector) and the previous hidden state, updates the hidden state, and optionally produces an output. The same RNN cell and parameters are used at every step.

Example: Processing the sentence "The cat sat on the mat"

The RNN processes each word in sequence, updating the hidden state as it moves through the sentence.



Step-by-step explanation

1. **Word representation**
   Each word is converted to a vector (e.g. using word embeddings).
2. **Update hidden state**
   The RNN takes the current word vector x_t and the previous hidden state h_{t-1} to compute the new hidden state h_t.
3. **Produce output (optional)**
   At each step, the model can produce an output y_t (e.g. a class score or the next word probability).
4. **Move to the next word**
   The new hidden state h_t is passed to the next time step.
5. **Final hidden state**
   After the last word, the final hidden state h_6 contains information about the whole sentence.

What does the hidden state contain?

- A compressed representation of all the words seen so far.
- It is updated at every time step to incorporate new information.
- It helps the model keep track of context (e.g. subject, tense, meaning).
- The final hidden state summarises the sentence and can be used for tasks such as text classification.

Analogy

The hidden state is like a running note in our mind. As we read each word, we update our understanding based on what we have already read.

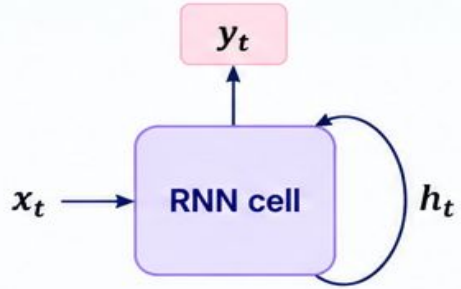
Key takeaway

An RNN processes a sentence one word at a time, using a hidden state to remember and integrate information from previous words.

Unrolling an RNN Through Time

An RNN can be viewed as a single cell repeated over multiple time steps. Unrolling the RNN through time shows how the same network (with shared parameters) is applied at each step in the sequence.

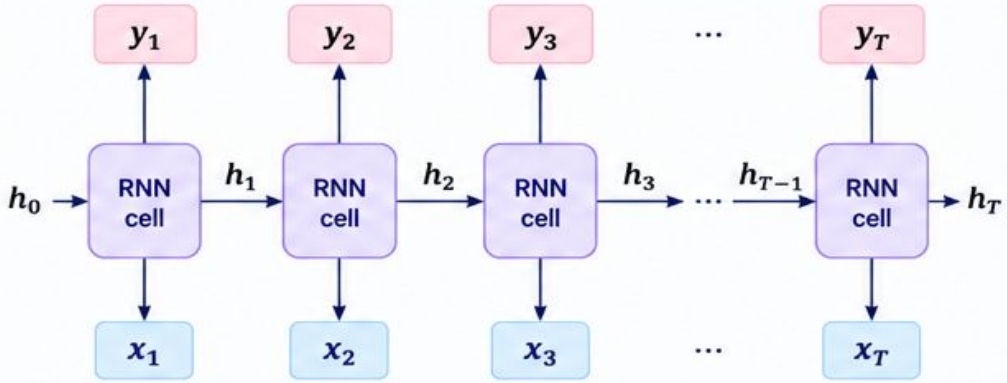
Recurrent view (compact form)



$$h_t = f(h_{t-1}, x_t)$$
$$y_t = g(h_t)$$

Unroll through time

Unrolled view (explicit time steps)



The same RNN cell and parameters are used at every time step.

Key points

1. **Equivalent representations**
   The unrolled RNN is mathematically equivalent to the compact recurrent form.
2. **Same parameters at each step**
   The same weights (W_{xh}, W_{hh}, W_{hy}) are used for all time steps.
3. **Reveals the computation graph**
   Unrolling shows how information flows through time from x_1 to x_T.
4. **Useful for training**
   It allows us to apply backpropagation through time (BPTT) by computing gradients across all time steps.

Parameter sharing

Each unrolled RNN cell uses the same set of parameters:

- W_{xh} – input to hidden weights
- W_{hh} – hidden to hidden (recurrent) weights
- W_{hy} – hidden to output weights
- b_h, b_y – bias vectors

The parameters do not depend on the time step t, which allows the model to handle sequences of any length.

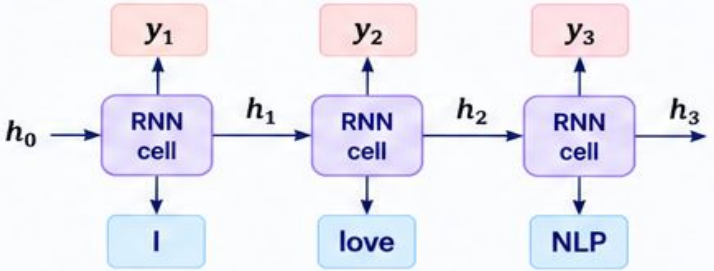
Intuition

Unrolling an RNN is like looking at a video frame by frame. The same "thinking mechanism" (RNN cell) is applied at every frame, carrying forward a memory of what has happened so far.

*"Unrolling helps us see the **sequence of computations** that happen over time, even though the **RNN** is the **same** single cell **repeated**."*

Example: Short sentence

Sentence: "I love NLP"



RNN Computation at One Time Step

At each time step t, the RNN takes the current input vector x_t and the previous hidden state h_{t-1}, computes a new hidden state h_t using a non-linear transformation, and optionally produces an output y_t.

1 Inputs at time step t

x_t	Current input vector (e.g. word embedding)
h_{t-1}	Previous hidden state (memory from earlier words)

2 RNN computation

Hidden state update

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b_h\right)$$

Output (if required)

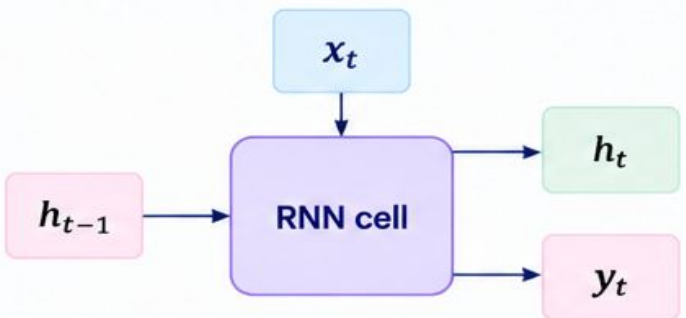
$$y_t = W_{hy}h_t + b_y$$

(followed by a softmax for classification)

3 Output at time step t

h_t	**New hidden state** (contains information about the current word and the past context)
y_t	**Output vector** (e.g. class scores, next word probabilities, or a feature vector)

Diagram



Meaning of the parameters

W_{xh}	Input-to-hidden weight matrix (size: $d_h \times d_x$)
W_{hh}	Hidden-to-hidden weight matrix (size: $d_h \times d_h$)
W_{hy}	Hidden-to-output weight matrix (size: $d_y \times d_h$)
b_h	Hidden bias vector (size: d_h)
b_y	Output bias vector (size: d_y)
tanh	Non-linear activation function (applied element-wise)

Numerical example (illustrative)

Suppose we have (for simplicity): $d_x = 3$, $d_h = 2$

Given:

- $x_t = [0.2, 0.5, -0.1]$
- $h_{t-1} = [0.1, -0.3]$
- W_{xh}, W_{hh}, b_h (predefined weights and biases)

Compute:

1. $z_t = W_{xh}x_t + W_{hh}h_{t-1} + b_h$
2. $h_t = \tanh(z_t)$
3. $y_t = W_{hy}h_t + b_y$ (if an output is required)

Result:
h_t is a new vector that summarises the information up to the current word.

Key Takeaway: The RNN computation at each time step combines the current input and the previous hidden state using a linear transformation and a non-linear activation (tanh) to produce a new hidden state, and optionally an output. The same parameters are used at every time step.



RNN — Worked Example

Let's see a simple numerical example to understand how an RNN processes a sentence step by step.
We will use a small vocabulary, low-dimensional vectors and simple weights for clarity.

1 Sentence and word embeddings

Sentence: I | love | NLP

Input vectors (x_t) (dimension = 3)

$$x_1 = \begin{bmatrix} 1 \\ 0 \\ 0 \end{bmatrix} \quad x_2 = \begin{bmatrix} 0 \\ 1 \\ 0 \end{bmatrix} \quad x_3 = \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix}$$

2 Model parameters

We use a small RNN with hidden size $h = 2$.

$$W_{xh} = \begin{bmatrix} 0.5 & 0.1 & -0.3 \\ 0.8 & -0.2 & 0.4 \end{bmatrix} \quad (2 \times 3)$$

$$W_{hh} = \begin{bmatrix} 0.7 & 0.2 \\ -0.1 & 0.5 \end{bmatrix} \quad (2 \times 2)$$

$$W_{hy} = \begin{bmatrix} 1.0 & -1.0 \end{bmatrix} \quad (1 \times 2)$$

$$b_y = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} \quad \text{(2 scalar)}$$

$$h_0 = \begin{bmatrix} 0.0 \\ 0.0 \end{bmatrix} \quad \text{(initial hidden state)}$$

3 Computation at each time step

$$h_t = \tanh\left(W_{xh}x_t + W_{hh}h_{t-1} + b_h\right)$$

$$y_t = W_{hy}h_t + b_y$$

$$\tanh(z) = \frac{e^z - e^{-z}}{e^z + e^{-z}} \quad \text{(applied element-wise)}$$

t	x_t (word)	h_t (new hidden state)	y_t (output)
1	I	$h_1 = \begin{bmatrix} 0.4621 \\ 0.6640 \end{bmatrix}$	$y_1 = -0.2019$
2	love	$h_2 = \begin{bmatrix} 0.6155 \\ 0.4022 \end{bmatrix}$	$y_2 = 0.2133$
3	NLP	$h_3 = \begin{bmatrix} -0.0454 \\ 0.4928 \end{bmatrix}$	$y_3 = -0.5382$

4 Step-by-step calculations

$t = 1$ (word: I)

$$z_1 = W_{xh}x_1 + W_{hh}h_0 + b_h = \begin{bmatrix} 0.5 \\ 0.8 \end{bmatrix}$$

$$h_1 = \tanh(z_1) = \begin{bmatrix} 0.4621 \\ 0.6640 \end{bmatrix}$$

$$y_1 = W_{hy}h_1 + b_y = -0.2019$$

$t = 2$ (word: love)

$$z_2 = W_{xh}x_2 + W_{hh}h_1 + b_h = \begin{bmatrix} 0.7175 \\ 0.4264 \end{bmatrix}$$

$$h_2 = \tanh(z_2) = \begin{bmatrix} 0.6155 \\ 0.4022 \end{bmatrix}$$

$$y_2 = W_{hy}h_2 + b_y = \mathbf{0.2133}$$

$t = 3$ (word: NLP)

$$z_3 = W_{xh}x_3 + W_{hh}h_2 + b_h = \begin{bmatrix} -0.0455 \\ 0.5389 \end{bmatrix}$$

$$h_3 = \tanh(z_3) = \begin{bmatrix} -0.0454 \\ 0.4928 \end{bmatrix}$$

$$y_3 = W_{hy}h_2 + b_y = -0.5382$$

5 Interpretation

- The hidden state h_t changes at each time step, capturing information from the current word and the previous context.
- After reading "I", the hidden state represents the meaning of the first word.
- After "love", the state is updated to include information about the phrase "I love".
- After "NLP", the final hidden state h_3 summarises the whole sentence.
- The output y_t can be used for a task such as sentiment classification or next word prediction.

6 Key takeaway

An RNN processes a sentence step by step, updating its hidden state to capture the meaning of the sequence.

The same parameters are used at every time step, allowing the model to handle sequences of any length.

From Word Embeddings to RNN Inputs

An RNN does not take words directly as input. Each word is first converted into a numeric vector representation (word embedding), and these vectors are then fed to the RNN one at a time as the input sequence.

1 Words to vectors

Each word in the vocabulary is mapped to a dense vector of real numbers (an embedding) that captures its meaning and semantic similarity.

Word	Embedding (dimension = 5)
cat	[0.12 0.34 −0.27 0.08 0.15]
dog	[0.10 0.31 −0.25 0.05 0.12]
mat	[−0.21 0.09 0.33 −0.18 0.24]
love	[0.45 −0.12 0.08 0.31 −0.26]

Words with similar meanings (e.g. “cat” and “dog”) have similar vectors.

2 A sentence becomes a sequence of vectors

A sentence is converted into a sequence of word embeddings, which will be fed to the RNN one vector at a time.

Example sentence: **“The cat sat on the mat”**

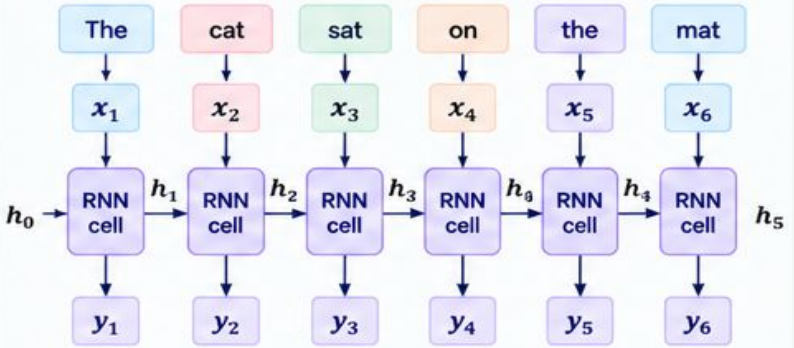
The	cat	sat	on	the	mat
0.11	0.12	0.03	−0.14	0.11	−0.21
−0.03	0.34	0.18	0.07	−0.03	0.09
0.25	−0.27	0.21	0.36	0.25	0.33
0.07	0.08	−0.11	0.02	0.07	−0.18
−0.19	0.15	0.09	−0.08	−0.19	0.24

Input sequence to the RNN

$$x_1, x_2, x_3, x_4, x_5, x_6$$

3 Feeding the sequence to the RNN

Each word embedding x_t is used as the input at time step t. The RNN processes the sequence in order, updating its hidden state at each step.



4 How are word embeddings obtained?

Word embeddings can be:

- Pre-trained embeddings (e.g. Word2Vec, GloVe, FastText)
- Contextual embeddings (e.g. from BERT, RoBERTa)
- Learned embeddings during training (randomly initialised and updated with the task)

In all cases, the RNN receives a sequence of fixed-size vectors as input.

Key points

1. Words must be converted to numeric vectors.
2. A sentence becomes a sequence of word embeddings.
3. Each embedding is fed to the RNN one at a time.
4. All input vectors have the same dimension (d).

Example: Input matrix form

The input to the RNN can be represented as a matrix X of shape $(T \times d)$, where T is the sentence length and d is the embedding dimension.

$$X = \begin{bmatrix} x_1^T \\ x_2^T \\ \vdots \\ x_T^T \end{bmatrix}$$

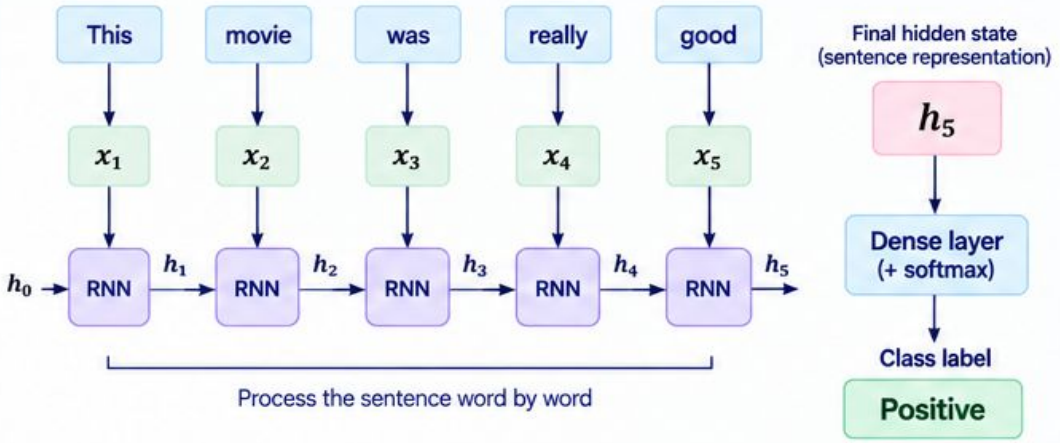
Each row corresponds to a word embedding. This matrix is processed sequentially by the RNN.

Key Takeaway: An RNN takes a sequence of word embeddings as input, processing one vector at a time to capture the meaning and context of the sentence.

RNN for Text Classification

In text classification, an RNN reads the words of a sentence one by one, updates its hidden state, and uses the final hidden state to predict a class label (e.g. positive or negative sentiment).

1 Overall architecture (many-to-one)



2 Steps in the classification process

1. **Convert words to embeddings**
   Each word is mapped to a dense vector x_t.
2. **Process the sequence with an RNN**
   The RNN updates its hidden state at each word, producing the final hidden state h_T.
3. **Use the final hidden state**
   h_T is a fixed-size vector summarising the meaning of the whole sentence.
4. **Predict the class**
   Pass h_T through a fully connected layer and a softmax to get class probabilities.
5. **Choose the label**
   The class with the highest probability is the predicted label.

3 Example: Sentiment classification

Sentence: **"This movie was really good"**

Predicted class probabilities

Positive	**0.87**
Negative	0.10
Neutral	0.03

Predicted label: **Positive**

4 Alternative pooling methods

Instead of using only the final hidden state, we can also:

Last hidden state (h_T)	Most common and works well in many cases.
Average pooling	Take the average of all hidden states.
Max pooling	Take the element-wise maximum of all hidden states.

All produce a fixed-size sentence representation that can be used for classification.

5 Training objective

We train the model using a labelled dataset and cross-entropy loss.

$$L = -\sum_{k} y_k \log(p_k)$$

- y_k : true label (one-hot encoded)
- p_k : predicted probability for class k

6 Key takeaway

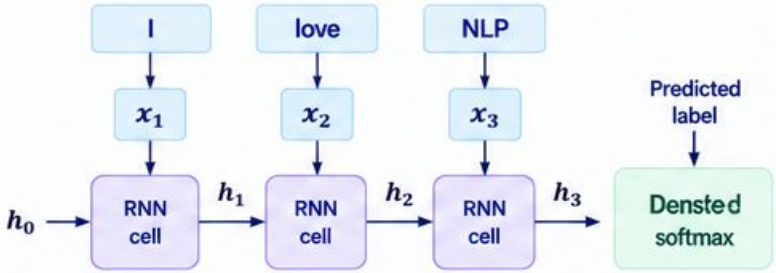
An RNN can be used for text classification by processing a sequence of word embeddings and using the final (or pooled) hidden state to predict a class label for the entire sentence.

How Does an RNN Learn?

An RNN learns by adjusting its parameters so that its predictions match the target outputs. This is done using backpropagation through time (BPTT) and gradient descent.

1 Forward pass: make a prediction

The RNN processes the input sequence word by word, updates its hidden state, and produces an output (e.g. a class prediction at the end of the sentence).



Same parameters (W_{xh}, W_{hh}, W_{hy}, b) are used at every time step.

2 Compute the loss

The prediction is compared with the true target label using a loss function (e.g. cross-entropy for classification).

Predicted probabilities	
Positive	0.70
Negative	0.20
Neutral	0.10

True label (one-hot)	
Positive	1
Negative	0
Neutral	0

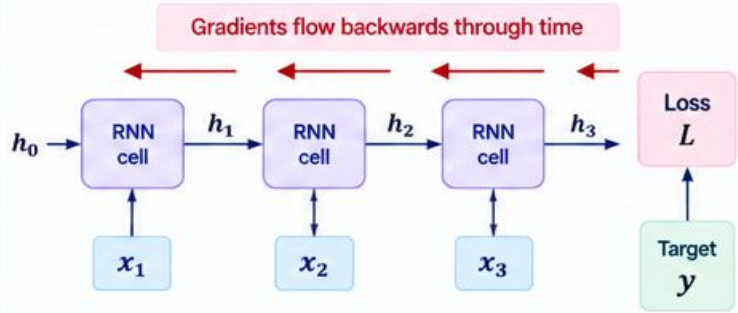
Cross-entropy loss

$$L = -\sum_k y_k \log(p_k)$$

y_k : true label
p_k : predicted probability

3 Backpropagation through time (BPTT)

The error is propagated backwards through all time steps to compute gradients of the loss with respect to the parameters.



4 Update the parameters

The parameters are updated using gradient descent (or a variant such as Adam).

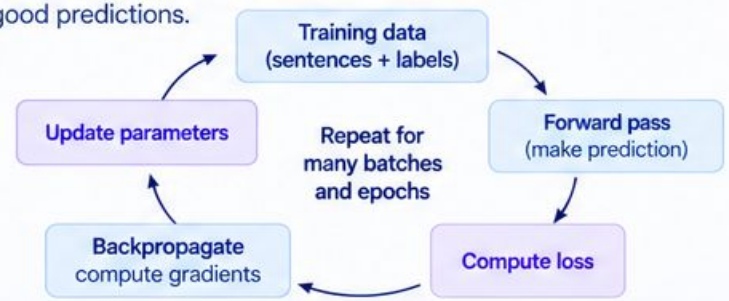
$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

where:

- θ are the model parameters (W_{xh}, W_{hh}, W_{hy}, b)
- η is the learning rate
- $\frac{\partial L}{\partial \theta}$ are the gradients computed using BPTT

5 Repeat for many examples

The process is repeated for many training sentences over multiple epochs until the model learns to make good predictions.



6 Key takeaways

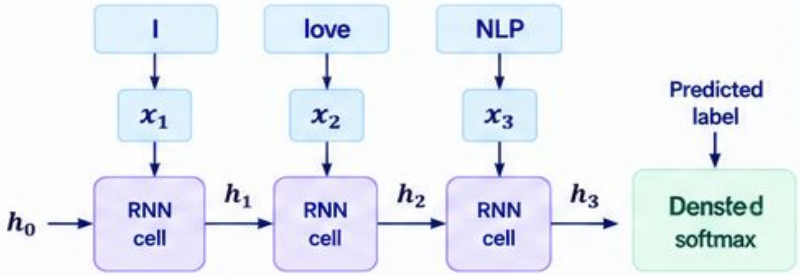
1. An RNN learns by minimising a loss function on a training dataset.
2. Gradients are computed using backpropagation through time (BPTT).
3. The same parameters are updated based on errors from all time steps in the sequence.
4. Training is done over many examples and epochs using gradient descent (e.g. SGD, Adam).
5. After training, the RNN can generalise to new sentences and make accurate predictions.

How Does an RNN Learn?

An RNN learns by adjusting its parameters so that its predictions match the target outputs. This is done using backpropagation through time (BPTT) and gradient descent.

1 Forward pass: make a prediction

The RNN processes the input sequence word by word, updates its hidden state, and produces an output (e.g. a class prediction at the end of the sentence).



Same parameters (W_{xh}, W_{hh}, W_{hy}, b) are used at every time step.

2 Compute the loss

The prediction is compared with the true target label using a loss function (e.g. cross-entropy for classification).

Predicted probabilities	
Positive	0.70
Negative	0.20
Neutral	0.10

True label (one-hot)	
Positive	1
Negative	0
Neutral	0

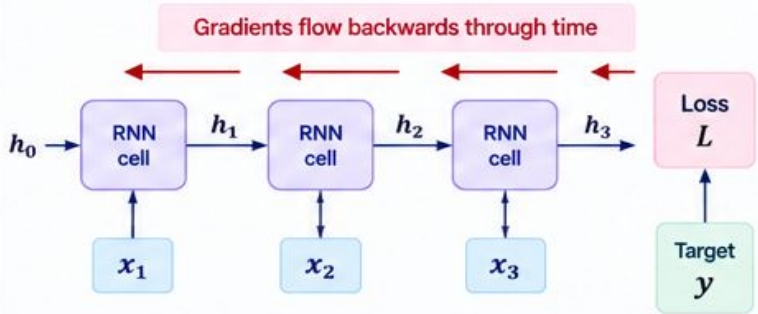
Cross-entropy loss

$$L = -\sum_k y_k \log(p_k)$$

y_k : true label
p_k : predicted probability

3 Backpropagation through time (BPTT)

The error is propagated backwards through all time steps to compute gradients of the loss with respect to the parameters.



4 Update the parameters

The parameters are updated using gradient descent (or a variant such as Adam).

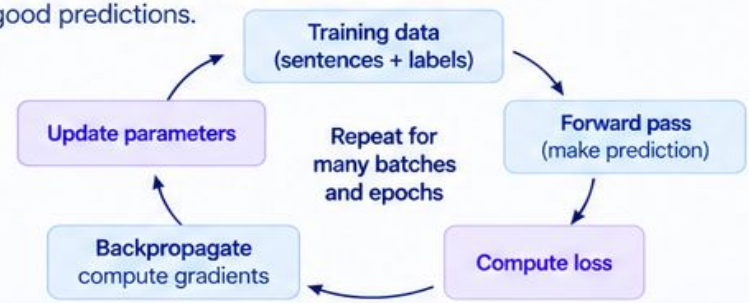
$$\theta \leftarrow \theta - \eta \frac{\partial L}{\partial \theta}$$

where:

- θ are the model parameters (W_{xh}, W_{hh}, W_{hy}, b)
- η is the learning rate
- $\frac{\partial L}{\partial \theta}$ are the gradients computed using BPTT

5 Repeat for many examples

The process is repeated for many training sentences over multiple epochs until the model learns to make good predictions.



6 Key takeaways

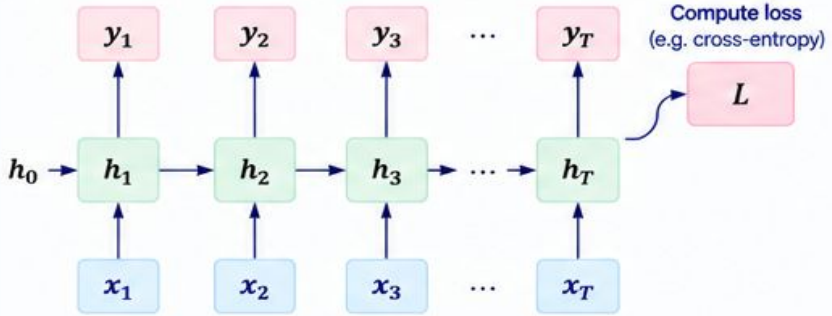
1. An RNN learns by minimising a loss function on a training dataset.
2. Gradients are computed using backpropagation through time (BPTT).
3. The same parameters are updated based on errors from all time steps in the sequence.
4. Training is done over many examples and epochs using gradient descent (e.g. SGD, Adam).
5. After training, the RNN can generalise to new sentences and make accurate predictions.

Backpropagation Through Time (BPTT)

BPTT is the algorithm used to train RNNs. It unrolls the network through time, computes the loss, and propagates the error gradients backwards through all time steps to update the shared parameters.

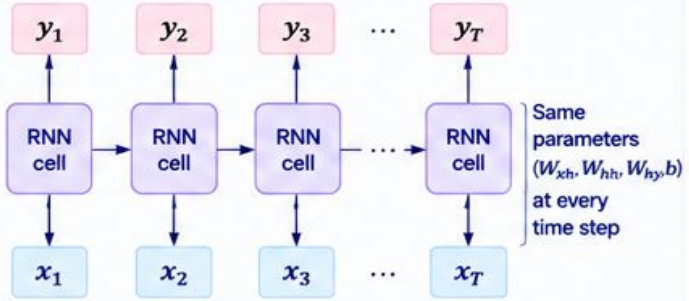
1 Forward pass (unroll the RNN)

Process the input sequence and store the hidden states at each time step.



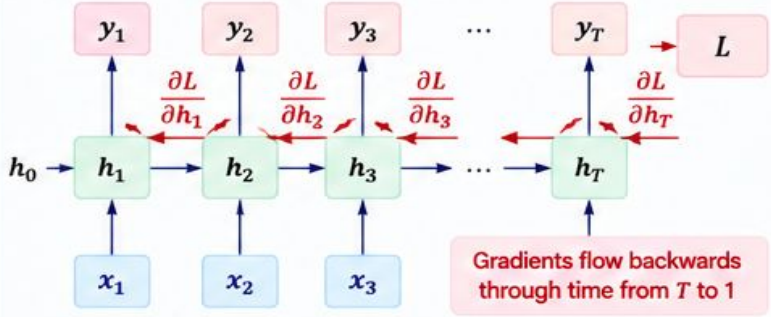
2 Unrolled RNN

The RNN is unfolded into a deep feedforward network with T time steps. The same parameters are used at every time step.



3 Backward pass (backpropagate through time)

Compute the gradient of the loss with respect to the parameters by propagating the error backwards from time step T to 1.



4 Gradients and parameter updates

At each time step, we compute gradients using the chain rule. The total gradient for each parameter is the sum of gradients from all time steps.

Gradient computation (example for W_{hh})

$$\frac{\partial L}{\partial W_{hh}} = \sum_{t=1}^{T} \frac{\partial L}{\partial h_t} \frac{\partial h_t}{\partial W_{hh}}$$

Parameter update (e.g. SGD)

$$W_{hh} \leftarrow W_{hh} - \eta \frac{\partial L}{\partial W_{hh}}$$

Similarly for W_{xh}, W_{hy}, b

5 Key points about BPTT

1. Unroll the RNN through time.
2. Compute the loss over the whole sequence (or at each time step).
3. Backpropagate the error from the last time step to the first using the chain rule.
4. Accumulate gradients for each parameter across all time steps.
5. Update the parameters using gradient descent (e.g. SGD, Adam).
6. In practice, we often use truncated BPTT (unroll for a fixed number of steps) for efficiency.

Truncated BPTT

For long sequences, we unroll the RNN for a fixed number of steps k (e.g. 20–50) instead of the full sequence.



Compute gradients over k steps, then continue with the next segment.

Key Takeaway: BPTT trains RNNs by propagating error gradients backwards through time, allowing the model to learn from the entire sequence.

The Vanishing Gradient Problem

In standard RNNs, gradients can become very small as they are propagated backward through many time steps, making it difficult for the model to learn long-term dependencies.

1 Why gradients vanish in RNNs?

During backpropagation through time, the gradient at an earlier time step is the product of many Jacobian terms.
If the magnitude of these terms is less than 1, the gradient shrinks exponentially as we go further back in time.

$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$

If $\left\| \frac{\partial h_k}{\partial h_{k-1}} \right\| < 1$, then $\left\| \frac{\partial L}{\partial h_t} \right\|$ becomes very small for small t (early time steps).

2 Intuition: repeated multiplication

In a simple RNN, the hidden state is computed as $h_t = \tanh(W_{xh}x_t + W_{hh}h_{t-1} + b_h)$.
The derivative of tanh is $1 - \tanh^2(z)$, which is always between 0 and 1. Repeated multiplication of values less than 1 leads to exponentially small gradients.

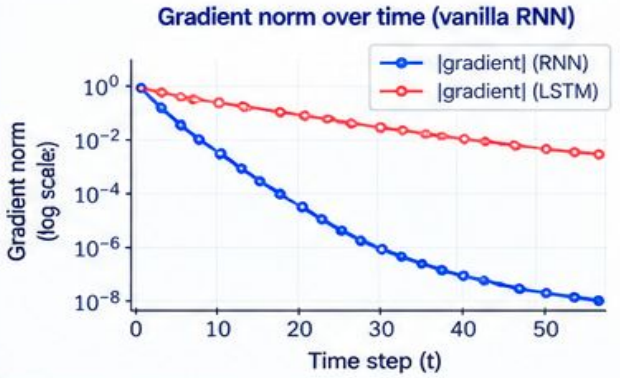
Derivative of tanh:

$$\frac{d}{dz}\tanh(z) = 1 - \tanh^2(z) \in (0, 1)$$

As the error is backpropagated through many time steps, the gradient becomes smaller and smaller, making it hard for the model to learn long-term dependencies.

3 Example: gradient decay over time

The plot below shows how the gradient norm typically decreases as it is propagated to earlier time steps in a vanilla RNN.

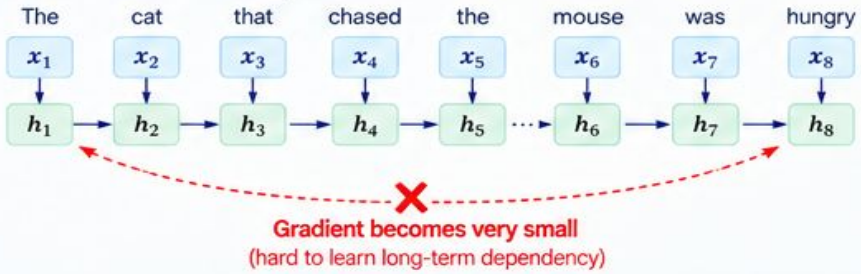


In a vanilla RNN, the gradient norm decreases exponentially, making it difficult to learn information from early time steps.

LSTM (and GRU) help to mitigate this problem.

4 Effect on learning long-term dependencies

Because gradients from distant time steps become very small, the model struggles to learn patterns that depend on information from far back in the sequence.



5 Real-world impact

- Difficult to capture long-range dependencies (e.g. subject–verb agreement, coreference, context across sentences).
- The model tends to focus more on recent words and forgets earlier information.
- Performance suffers on tasks requiring long-term context (e.g. document classification, machine translation).
- This limitation motivated the development of LSTM and GRU, which use gating mechanisms to preserve gradients over longer sequences.

6 How to address the vanishing gradient problem

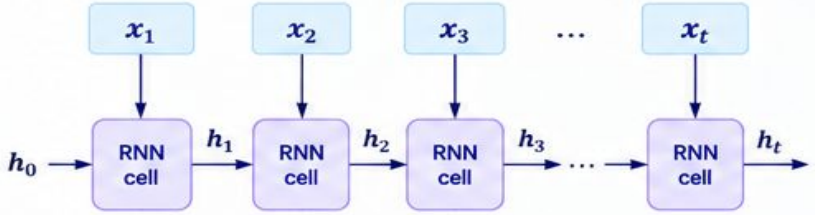
1. **Use LSTM or GRU**
   Gating mechanisms help preserve gradients over long sequences.
2. **Careful weight initialisation**
   e.g. orthogonal or identity initialisation for recurrent weights.
3. **Gradient clipping**
   Prevents exploding gradients (often used together with LSTM/GRU).
4. **Use shorter sequences**
   Truncate backpropagation through time (TBPTT).
5. **Alternative architectures**
   Consider attention mechanisms (e.g. Transformers) for very long-range dependencies.

Why RNNs Struggle with Long-Term Dependencies

Standard RNNs find it difficult to remember information from many time steps in the past because the influence of earlier inputs on the current hidden state and gradients decreases exponentially over time.

1 Information must pass through many steps

In an RNN, information from an earlier word has to pass through many recurrent steps to influence a later prediction. At each step, the hidden state is updated, which can dilute the original information.



Information from x_1 must pass through $t-1$ steps to affect h_t.

2 Repeated multiplication shrinks the signal

The hidden state update in a vanilla RNN is:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$$

When we unroll this, h_t depends on h_0 (and earlier inputs) through a product of weight matrices and tanh derivatives. If the spectral norm (or eigenvalues) of W_h is less than 1, the signal from earlier steps decays exponentially.

Simplified view (linear case)

$$h_t \approx W_h^t h_0 + \cdots$$

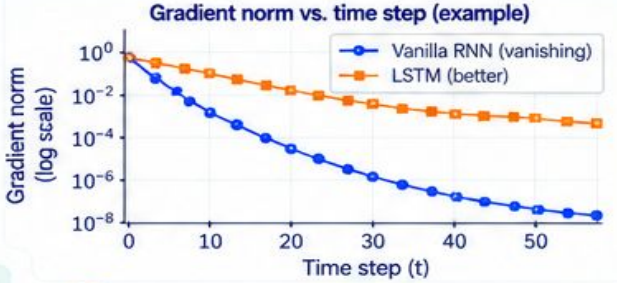
If $\|W_h\| < 1$, then $\|W_h^t\| \to 0$ as t increases.

In practice, because $\tanh'(z) \in (0, 1)$, the gradients also shrink exponentially during backpropagation.

3 Gradient vanishing over time

During backpropagation through time (BPTT), the gradient at an earlier time step is a product of many Jacobian terms. Each term typically has a magnitude less than 1, so the gradient becomes very small for distant time steps.

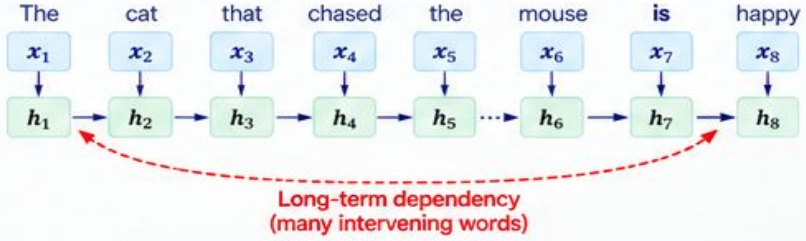
$$\frac{\partial L}{\partial h_t} = \frac{\partial L}{\partial h_T} \prod_{k=t+1}^{T} \frac{\partial h_k}{\partial h_{k-1}}$$



In a vanilla RNN, the gradient norm decreases rapidly over time, making it hard to learn long-term dependencies.

4 Example: subject–verb agreement

To correctly predict "is", the model needs to remember "The cat" even though it is several words back.



5 Key reasons RNNs struggle

1. Gradients vanish over long sequences due to repeated multiplication of small values.
2. Early information gets diluted as it passes through many time steps.
3. The model tends to focus on recent inputs, which are easier to learn.
4. Tanh activations squash the hidden state, limiting the range of values and gradients.
5. Training becomes difficult on long sequences, leading to poor performance on tasks that require long-range context.

6 How to address this problem

- Use gating mechanisms (LSTM, GRU) to preserve information over longer periods.
- Careful weight initialisation (e.g. orthogonal or identity initialisation).
- Gradient clipping to prevent exploding gradients.
- Use architectural improvements (e.g. attention mechanisms in Transformers) for very long-range dependencies.

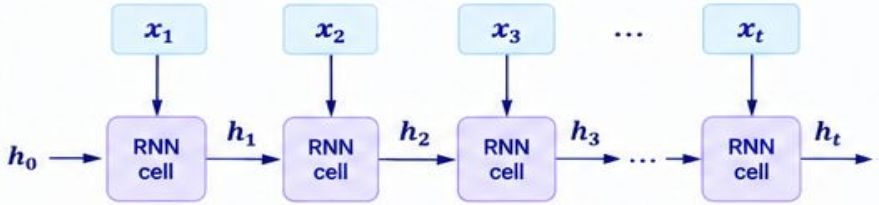
Key Takeaway: Vanilla RNNs struggle with long-term dependencies because both the hidden state and the gradients shrink exponentially over time, making it hard to retain and use information from distant parts of the sequence.

From RNN to LSTM — Why Do We Need Something Better?

Standard RNNs are simple and effective for short sequences, but they struggle to remember information over long periods. LSTMs were designed to overcome this limitation by using a more sophisticated memory mechanism.

1 Vanilla RNN: simple but limited

A vanilla RNN updates a single hidden state at each time step. While it works well for short sequences, it struggles to retain information from many time steps in the past.



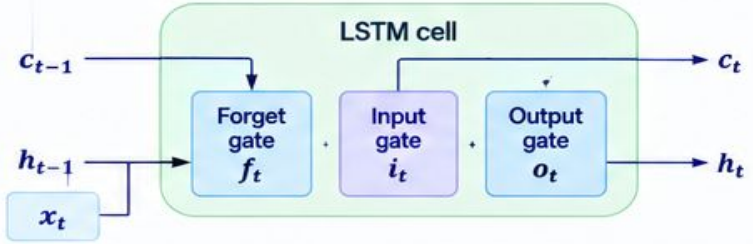
Problem: as information passes through many time steps, important signals can vanish, making it hard to learn long-term dependencies.

2 Why do RNNs struggle?

- **Vanishing gradients**
  During training, gradients become very small for early time steps, so the model cannot learn to keep important information for long.
- **Limited memory**
  The single hidden state has to carry all information, which can be easily overwritten by new inputs.
- **Short-term focus**
  RNNs tend to focus on recent inputs, making them less effective on long sequences (e. g. long sentences, documents).
- **Difficult to learn long-range dependencies**
  Tasks such as coreference resolution, document classification or long-form generation require information from far apart in the sequence.

3 LSTM: a better memory mechanism

LSTMs introduce a cell state and gating mechanisms to control what information to keep, forget, and output.



The gates allow the LSTM to selectively remember or forget information, helping it capture long-term dependencies more effectively.

4 RNN vs LSTM: key differences

Aspect	Vanilla RNN	LSTM
Memory	Single hidden state h_t	Hidden state h_t and cell state c_t
Long-term dependencies	Struggles (vanishing gradients)	Handles long-term dependencies using gates
Architecture complexity	Simple	More complex (gates and cell state)
Parameters	Fewer	More (3–4 gates)
Performance on long sequences	Often poor	Generally much better

5 Key takeaways

1. Vanilla RNNs are simple but struggle with long-term dependencies due to vanishing gradients.
2. LSTMs introduce a cell state and gates to control information flow.
3. The gating mechanism helps preserve important information over longer sequences.
4. LSTMs are more complex but perform much better on many NLP tasks.
5. LSTMs (and GRUs) are widely used when modelling long sequences of text.

6 Real-world NLP examples

- **Document classification**
  Use information from across a long document to predict the topic.
- **Machine translation**
  Remember context from earlier words in a sentence.
- **Dialogue systems**
  Keep track of context across multiple turns in a conversation.

Main Takeaway: We need something better than vanilla RNNs because they struggle to remember information over long sequences. LSTMs (and GRUs) address this with gating mechanisms that preserve important information, enabling better performance on real-world NLP tasks.

What is an LSTM?

An LSTM (Long Short-Term Memory) is a type of recurrent neural network designed to overcome the vanishing gradient problem by using a memory cell and gating mechanisms to control the flow of information over time.

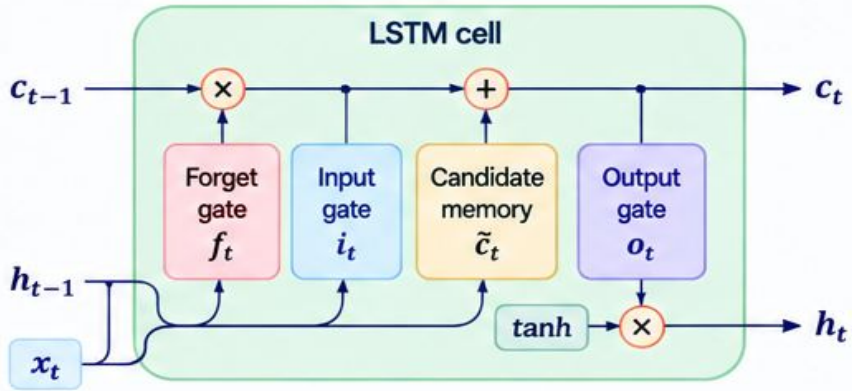
1 Key idea

An LSTM introduces a cell state that acts as a memory, and gates that decide what information to keep, forget, and output at each time step.

- Keeps important information for long periods
- Forgets irrelevant information
- Controls what information to output
- Helps the model learn long-term dependencies

2 LSTM cell architecture

An LSTM cell has three gates — forget, input and output — that control the flow of information.



The gates allow the LSTM to selectively remember or forget information, helping it capture long-term dependencies.

3 LSTM equations

At each time step t, the following computations are made:

Forget gate
$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$
Decides what information to forget from the cell state.

Input gate
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
Decides what new information to store.

Candidate memory
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$
Creates new candidate values to add to the cell state.

Update cell state
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
Combines old and new information.

Output gate
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
Decides what information to output.

Hidden state
$$h_t = o_t \odot \tanh(c_t)$$
Produces the hidden state for the next time step.

σ: sigmoid function tanh: hyperbolic tangent $\odot$: element-wise multiplication

4 How LSTMs help

- The cell state allows gradients to flow through many time steps, reducing the vanishing gradient problem.
- LSTMs can capture long-term dependencies in sequences (e.g. context in a sentence).
- They are particularly effective for tasks such as language modelling, machine translation and text classification.

5 Example intuition

"The cat that chased the mouse was hungry."

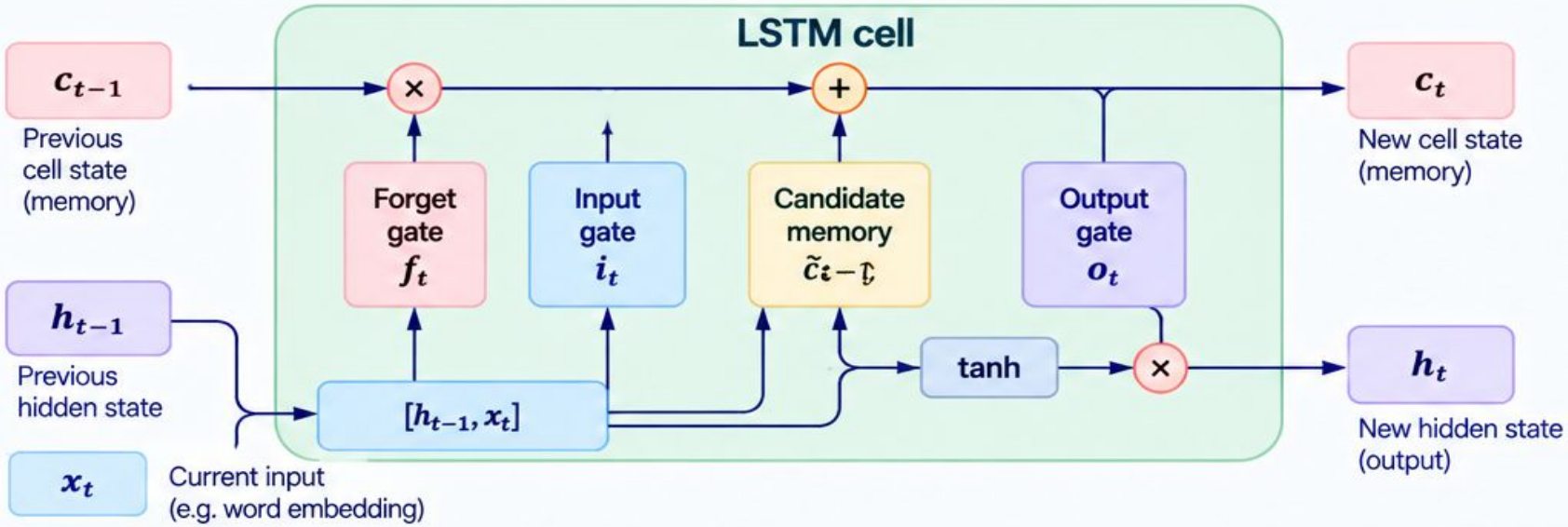
To correctly understand "was hungry" refers to "cat", the model needs to remember information from several words back. An LSTM can keep this information in its cell state.

6 Key takeaway

An LSTM is a more advanced RNN architecture that uses a **memory cell** and gating mechanisms to overcome the vanishing gradient problem and effectively capture long-term dependencies in sequential data.

The LSTM Cell — Basic Architecture

An LSTM cell uses a memory cell and three gates — forget, input and output — to control the flow of information over time. The gates decide what to forget, what new information to store, and what to output.



Operations legend

- × Element-wise multiplication
- \+ Element-wise addition
- σ Sigmoid activation (outputs between 0 and 1)
- tanh Tanh activation (outputs between −1 and 1)
- [,] Concatenation (along feature dimension)
- → Data flow

1 Forget gate

Decides what information to forget from the previous cell state.

$$f_t = \sigma\left(W_f\left[h_{t-1}, x_t\right] + b_f\right)$$

Values close to 0 forget information; values close to 1 keep information.

2 Input gate

Decides what new information to store in the cell state.

$$i_t = \sigma\left(W_i\left[h_{t-1}, x_t\right] + b_i\right)$$
$$\tilde{c}_t = \tanh\left(W_c\left[h_{t-1}, x_t\right] + b_c\right)$$

i_t controls how much new information to add. $\tilde{c}_t$ are the candidate values.

3 Update cell state

Combine the old cell state and the new candidate information.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

Keep some of the previous information and add new information.

4 Output gate

Decides what part of the cell state to output.

$$o_t = \sigma\left(W_o\left[h_{t-1}, x_t\right] + b_o\right)$$
$$h_t = o_t \odot \tanh\left(c_t\right)$$

The hidden state h_t is the output of the LSTM cell and is passed to the next time step.

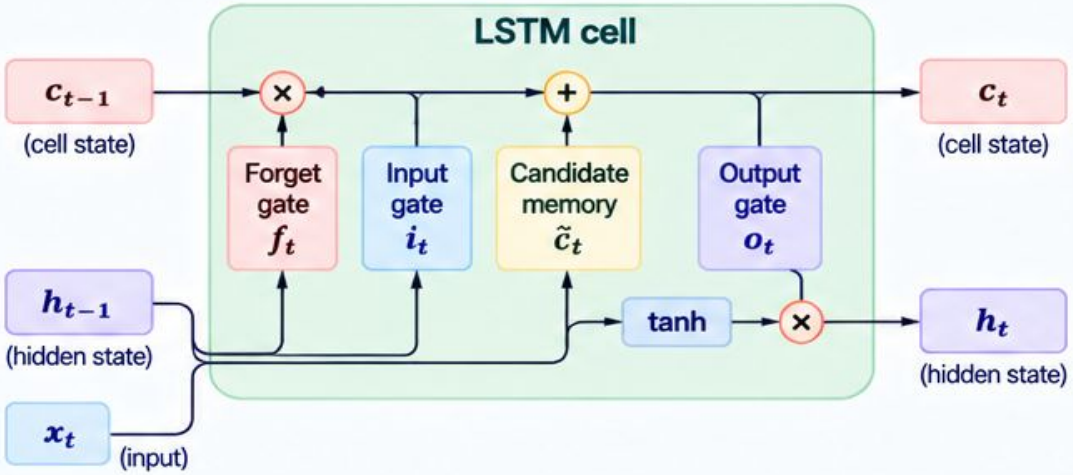
5 Key takeaway

The LSTM cell uses gates to control information flow, allowing it to remember important information over long sequences and overcome the vanishing gradient problem.

Hidden State and Cell State

An LSTM maintains two types of state: the hidden state (h_t) and the cell state (c_t). They serve different purposes and work together to allow the model to capture both short-term and long-term dependencies.

1 Where they appear in the LSTM cell



2 Cell state (c_t) — long-term memory

- The cell state c_t acts as a long-term memory that runs through the LSTM cell with minimal modification.
- Information can flow along the cell state for many time steps, which helps preserve important information over long sequences.
- The forget and input gates control what information is removed or added to the cell state.
- The cell state is mainly responsible for capturing long-term dependencies.

3 Hidden state (h_t) — short-term memory

- The hidden state h_t is the output of the LSTM cell at time step t.
- It is a short-term representation of the current context, combining information from the current input and the cell state.
- The hidden state is influenced by the output gate, which decides what part of the cell state to expose.
- The hidden state is used for making predictions (e.g. next word, class label) and is passed to the next time step.

4 Key differences

Aspect	Cell state (c_t)	Hidden state (h_t)
Purpose	Long-term memory	Short-term memory / output
Information flow	Flows with minimal modification	Updated at each step
Controlled by	Forget and input gates	Output gate
Used for	Storing information (long-term dependencies)	Making predictions and passed to next step

5 Analogy

Cell state:
Like a long-term notebook where important information is kept over a long time.

Hidden state:
Like a summary of the current page, used to make decisions right now.

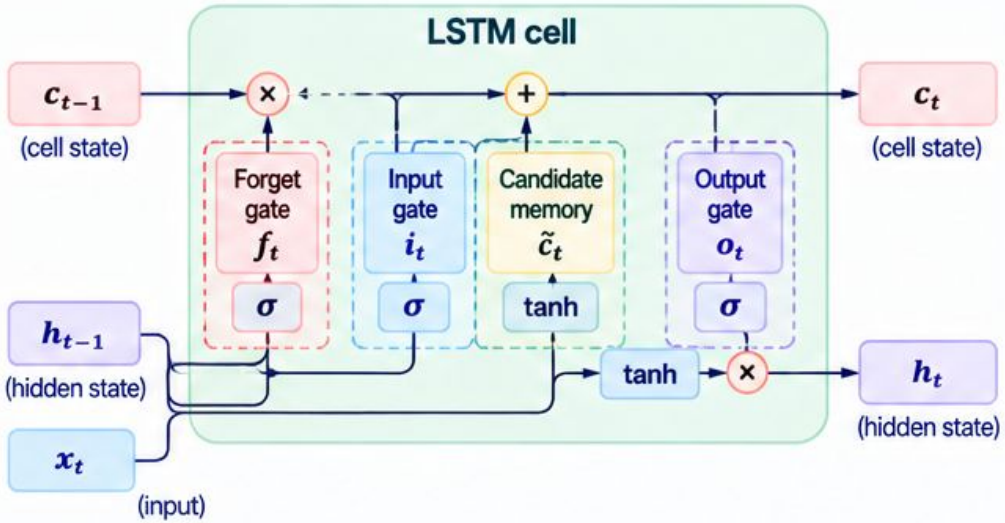
6 Key takeaway

The cell state helps the LSTM remember important information over long periods, while the hidden state represents the current context and is used for making predictions. Together, they allow the LSTM to model both long-term and short-term dependencies effectively.

The Three LSTM Gates — Forget, Input & Output

An LSTM uses three gates — forget, input and output — to control the flow of information. Each gate looks at the current input x_t and the previous hidden state h_{t-1}, and outputs a value between 0 and 1 to decide what to forget, what to store, and what to output.

1 LSTM cell with three gates



2 Forget gate (f_t)

Decides what information to forget from the previous cell state.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

f_t outputs values between 0 and 1.

- 0 → forget (remove information)
- 1 → keep (retain information)



3 Input gate (i_t)

Decides what new information to store in the cell state.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

i_t controls how much of the new candidate information $\tilde{c}_t$ to add.

- 0 → do not add
- 1 → add fully



4 Output gate (o_t)

Decides what part of the cell state to output as the hidden state.

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(c_t)$$

o_t controls how much of the cell state to expose as the hidden state.

- 0 → output little
- 1 → output fully



5 Update equations (summary)

The three gates work together to update the cell state and hidden state:

1. **Forget:**
$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

2. **Input:**
$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

3. **Update cell state:**
$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

4. **Output:**
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(c_t)$$

6 Intuition

Forget gate: "What should I forget?"
(Removes irrelevant information)

Input gate: "What should I add?"
(Stores new relevant information)

Output gate: "What should I show?"
(Controls what to output)

7 Key takeaway

The three gates allow the LSTM to selectively keep, update and expose information, enabling it to capture long-term dependencies and overcome the vanishing gradient problem.



How Information Flows Through an LSTM

At each time step, the LSTM takes the current input and the previous hidden state, uses three gates to control the information flow, updates the cell state, and produces a new hidden state and output.

1 Inputs at time step t

The LSTM receives the current input x_t, the previous hidden state h_{t-1} and the previous cell state c_{t-1}.

- c_{t-1} (cell state) Long-term memory
- h_{t-1} (hidden state) Short-term memory
- x_t (current input) e.g. word embedding

2 Forget gate

Decides what information to forget from the previous cell state.

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$$

Values close to 0 forget information; values close to 1 keep information.

3 Input gate and candidate memory

Decides what new information to store in the cell state.

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$$
$$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$$

i_t controls how much new information to add.
$\tilde{c}_t$ contains the new candidate information.

4 Update cell state

Combine the kept information and the new candidate information.

$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

The new cell state c_t carries important information forward over time (long-term memory).

5 Output gate

Decides what part of the cell state to output as the hidden state.

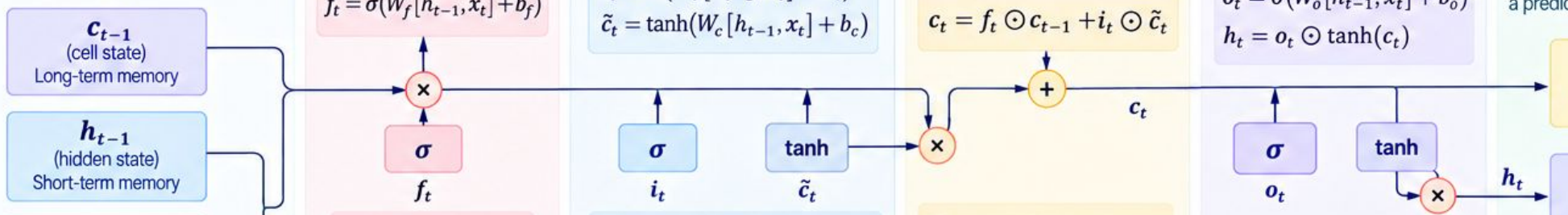
$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$$
$$h_t = o_t \odot \tanh(c_t)$$

The output gate controls how much of the cell state to expose as the hidden state.

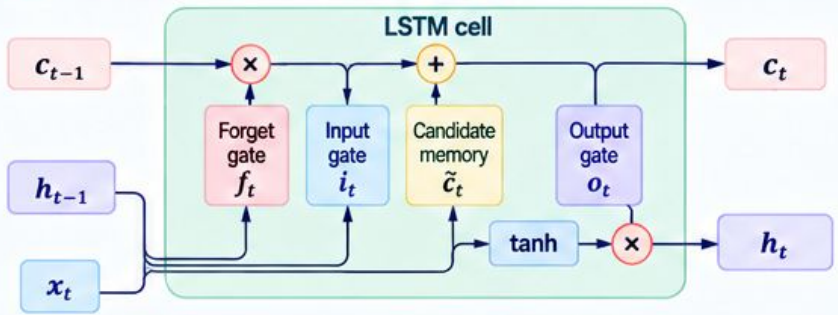
6 Outputs at time step t

The LSTM produces an updated cell state c_t and a new hidden state h_t, which can be used for the next time step or for making a prediction.

- c_t (updated cell state) (Long-term memory)
- h_t (new hidden state) (Short-term memory / output)



7 Overall LSTM cell diagram



8 Step-by-step flow summary

1. Take input x_t, previous hidden state h_{t-1} and cell state c_{t-1}.
2. Compute forget gate f_t and decide what to keep from c_{t-1}.
3. Compute input gate i_t and candidate memory $\tilde{c}_t$.
4. Update the cell state: $c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$.
5. Compute output gate o_t and new hidden state $h_t = o_t \odot \tanh(c_t)$.
6. Pass c_t and h_t to the next time step (and/or to a prediction layer).

9 Key takeaways

- Information flows through the cell state mainly unchanged, helping to capture long-term dependencies.
- The three gates control what to forget, what to add, and what to output.
- The hidden state represents the current context and is used for making predictions.
- This controlled flow of information helps LSTMs overcome the vanishing gradient problem and perform well on long sequences.

LSTM — Worked NLP Example

Let's see how an LSTM processes a sentence in a real NLP task. We use the example *"The cat that chased the mouse was hungry"* and show how the LSTM can remember the subject "cat" until the verb "was", even with intervening words.

NLP task
Subject–verb agreement:
Predict whether the verb matches the subject (grammatical or not).

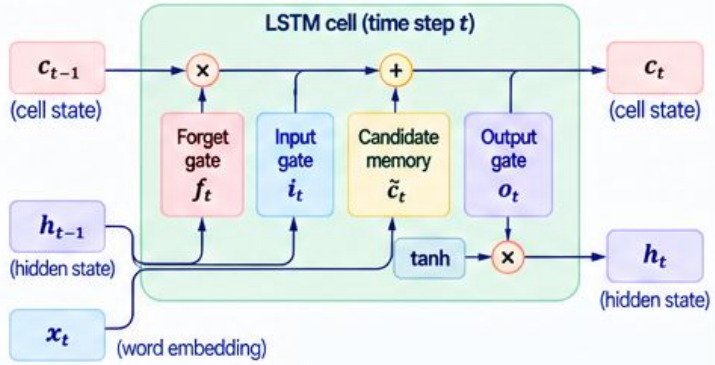
Input: The cat that chased the mouse was hungry
Output: Grammatical

1 Input sentence and word embeddings

	The	cat	that	chased	the	mouse	was	hungry
Words (x_t)								
Word embeddings (e.g. 50–300 dim)	0.12 −0.03 0.45 ...	0.67 0.21 −0.18 ...	0.05 0.33 −0.22 ...	−0.11 0.48 0.36 ...	0.09 −0.27 0.31 ...	0.62 0.14 −0.25 ...	−0.30 0.58 0.12 ...	0.28 −0.16 0.53 ...

Each word is converted to a vector (embedding) that captures its meaning.

2 LSTM cell at each time step



3 Gate equations (at each time step t)

Forget gate	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ Decides what to forget from c_{t-1}.
Input gate	$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ Decides what new information to add.
Candidate memory	$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$ Creates new candidate values.
Update cell state	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ Combines old and new information.
Output gate	$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$ Decides what to output.
Hidden state	$h_t = o_t \odot \tanh(c_t)$ Produces the hidden state for the next step.

4 Step-by-step illustration (conceptual values)

The LSTM keeps information about "cat" in the cell state, even after several intervening words, so it can correctly associate it with "was".

Time step (t)	1	2	3	4	5	6	7	8
Word (x_t)	The	cat	that	chased	the	mouse	was	hungry
Hidden state h_t (remembers context)	Start of sentence	Encodes "cat" (subject)	Still remembers "cat"	Adds verb "chased"	Keeps "cat" in memory	Adds object "mouse"	Matches with "cat" (subject-verb)	Completes meaning
Cell state c_t (long-term memory)	Initial state	Stores subject "cat"	Preserves "cat"	Keeps "cat" (despite new verb)	Still stores "cat"	Still stores "cat"	Uses "cat" to process "was"	Final state

5 Final prediction (sentence classification)

The final hidden state (or an aggregated representation) is passed to a dense layer for classification.



The model correctly predicts that the sentence is grammatical, because it remembered the subject "cat" until the verb "was".

6 Why this matters

- The LSTM's cell state allows it to remember important information (e.g. the subject "cat") over several time steps.
- This helps the model handle long-range dependencies, which are common in language.
- Without such a mechanism, a simple RNN might forget "cat" by the time it reaches "was".

7 Try it yourself

- Use a pre-trained LSTM (e.g. in PyTorch or TensorFlow) to classify sentences as grammatical or ungrammatical.
- Experiment with sentences like:
  - *The cats that chased the mouse were hungry.* (✓)
  - *The cat that chased the mouse were hungry.* (✗)

8 Key takeaway

LSTMs can keep important information in memory over long sequences, enabling better understanding and more accurate predictions in NLP tasks.

LSTM — Worked NLP Example

Let's see how an LSTM processes a sentence in a real NLP task. We use the example *"The cat that chased the mouse was hungry"* and show how the LSTM can remember the subject "cat" until the verb "was", even with intervening words.

NLP task
Subject-verb agreement:
Predict whether the verb matches the subject (grammatical or not).

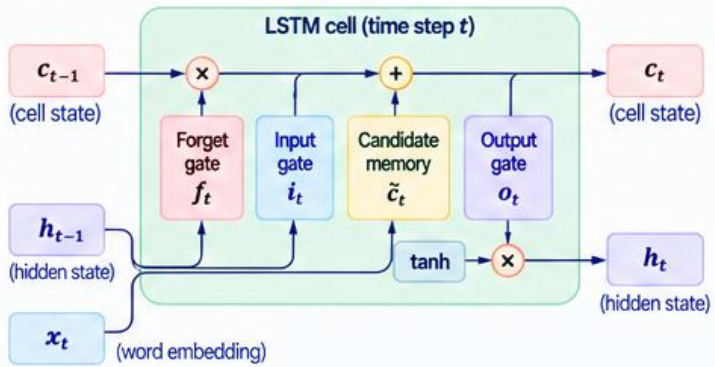
Input: The cat that chased the mouse was hungry
Output: Grammatical

1 Input sentence and word embeddings

	The	cat	that	chased	the	mouse	was	hungry
Words (x_t)								
Word embeddings (e.g. 50–300 dim)	0.12 −0.03 0.45 ...	0.67 0.21 −0.18 ...	0.05 0.33 −0.22 ...	−0.11 0.48 0.36 ...	0.09 −0.27 0.31 ...	0.62 0.14 −0.25 ...	−0.30 0.58 0.12 ...	0.28 −0.16 0.53 ...

Each word is converted to a vector (embedding) that captures its meaning.

2 LSTM cell at each time step



3 Gate equations (at each time step t)

Forget gate	$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$ Decides what to forget from c_{t-1}.
Input gate	$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$ Decides what new information to add.
Candidate memory	$\tilde{c}_t = \tanh(W_c[h_{t-1}, x_t] + b_c)$ Creates new candidate values.
Update cell state	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ Combines old and new information.
Output gate	$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$ Decides what to output.
Hidden state	$h_t = o_t \odot \tanh(c_t)$ Produces the hidden state for the next step.

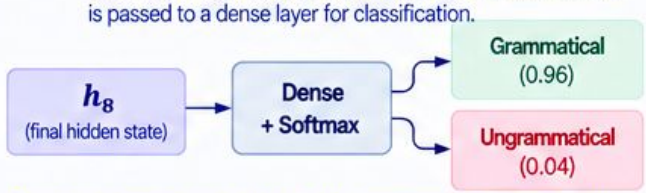
4 Step-by-step illustration (conceptual values)

The LSTM keeps information about "cat" in the cell state, even after several intervening words, so it can correctly associate it with "was".

Time step (t)	1	2	3	4	5	6	7	8
Word (x_t)	The	cat	that	chased	the	mouse	was	hungry
Hidden state h_t (remembers context)	Start of sentence	Encodes "cat" (subject)	Still remembers "cat"	Adds verb "chased"	Keeps "cat" in memory	Adds object "mouse"	Matches with "cat" (subject-verb)	Completes meaning
Cell state c_t (long-term memory)	Initial state	Stores subject "cat"	Preserves "cat"	Keeps "cat" (despite new verb)	Still stores "cat"	Still stores "cat"	Uses "cat" to process "was"	Final state

5 Final prediction (sentence classification)

The final hidden state (or an aggregated representation) is passed to a dense layer for classification.



The model correctly predicts that the sentence is grammatical, because it remembered the subject "cat" until the verb "was".

6 Why this matters

- The LSTM's cell state allows it to remember important information (e.g. the subject "cat") over several time steps.
- This helps the model handle long-range dependencies, which are common in language.
- Without such a mechanism, a simple RNN might forget "cat" by the time it reaches "was".

7 Try it yourself

- Use a pre-trained LSTM (e.g. in PyTorch or TensorFlow) to classify sentences as grammatical or ungrammatical.
- Experiment with sentences like:
  - *The cats that chased the mouse were hungry.* (✓)
  - *The cat that chased the mouse were hungry.* (✗)

8 Key takeaway

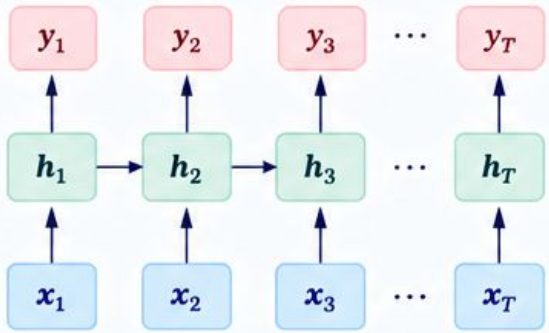
LSTMs can keep important information in memory over long sequences, enabling better understanding and more accurate predictions in NLP tasks.

RNN vs LSTM

LSTM is an improved version of the basic RNN that can learn and remember information over longer sequences.

RNN (Recurrent Neural Network)

A simple recurrent architecture that maintains a single hidden state.

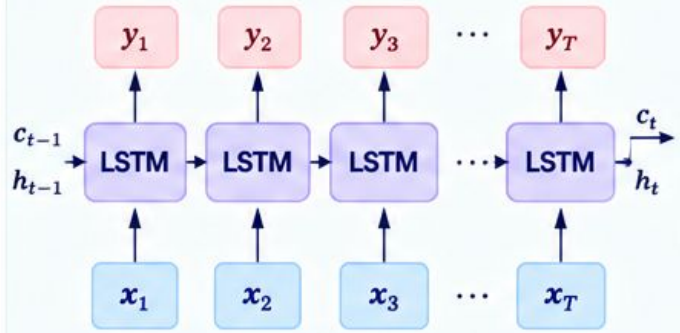


$$h_t = \tanh(W_{hh}h_{t-1} + W_{xh}x_t + b_h)$$
$$y_t = W_{hy}h_t + b_y$$

- Single hidden state h_t
- Information is passed from one step to the next
- Simple architecture
- Struggles with long-term dependencies due to the vanishing gradient problem

LSTM (Long Short-Term Memory)

An enhanced recurrent architecture with memory cell and three gates.



$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
$$h_t = o_t \odot \tanh(c_t)$$

- Two states: cell state c_t (long-term memory) and hidden state h_t (short-term)
- Three gates: forget (f_t), input (i_t), output (o_t)
- Better at capturing long-term dependencies
- More complex, but widely used in NLP

Aspect	RNN	LSTM
Architecture	Single hidden state h_t	Two states: hidden state h_t and cell state c_t with three gates
Memory	Short-term memory	Both short-term (h_t) and long-term memory (c_t)
Handling long sequences	Struggles (vanishing gradient problem)	Designed to overcome vanishing gradients
Gates	None	Forget gate, input gate, output gate
Ability to remember information	Limited to recent context	Can remember important information over many time steps
Parameters	Fewer parameters (simpler)	More parameters (more complex)
Training	Can be faster, but may not capture long-range dependencies	Slightly slower, but usually gives better performance on complex tasks
Typical applications	Simple or short sequences	Widely used in NLP (e.g. language modelling, machine translation, sentiment analysis)

Key takeaway

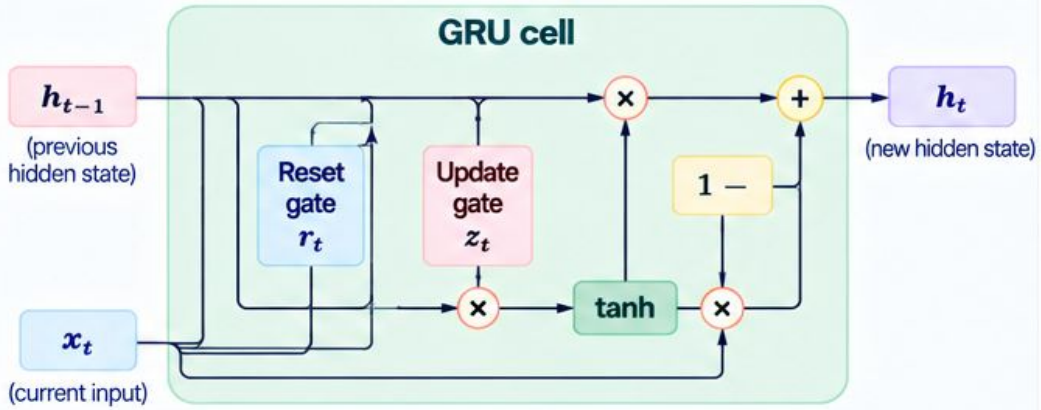
LSTMs extend the basic RNN with a memory cell and gating mechanisms, allowing them to retain important information over long sequences. This makes them much more effective than vanilla RNNs for real-world NLP tasks.

What is a GRU?

A Gated Recurrent Unit (GRU) is a simpler and more efficient variant of LSTM. It uses gating mechanisms to control information flow, but with fewer gates and no separate cell state, while still addressing the vanishing gradient problem and capturing long-term dependencies.

1 GRU architecture

A GRU cell uses two gates — a reset gate and an update gate — and maintains a single hidden state h_t.



2 GRU equations

Given input x_t and previous hidden state h_{t-1}:

Reset gate (r_t)

$$r_t = \sigma(W_r\,[h_{t-1}, x_t] + b_r)$$

Update gate (z_t)

$$z_t = \sigma(W_z\,[h_{t-1}, x_t] + b_z)$$

Candidate hidden state ($\tilde{h}_t$)

$$\tilde{h}_t = \tanh(W_h\,[r_t \odot h_{t-1}, x_t] + b_h)$$

New hidden state (h_t)

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

σ is the sigmoid function and $\odot$ denotes element-wise multiplication (Hadamard product).

3 How it works (intuition)

Reset gate (r_t)

- Controls how much of the past information to forget when computing the new candidate state $\tilde{h}_t$.
- r_t close to 0 → ignore past state.
  r_t close to 1 → use more past state.

Update gate (z_t)

- Decides how much of the new candidate state $\tilde{h}_t$ to use versus keeping the previous hidden state h_{t-1}.
- z_t close to 0 → use more new information.
  z_t close to 1 → keep more of the past state.

Candidate hidden state ($\tilde{h}_t$)

- Creates new content based on the current input x_t and the filtered past state (via r_t).

Final hidden state (h_t)

- A combination of the previous state and the new content, controlled by z_t.

4 GRU vs LSTM vs Vanilla RNN

Aspect	RNN	LSTM	GRU
Memory mechanism	Single hidden state	Hidden state + cell state	Single hidden state
Gates	None	Three gates (forget, input, output)	Two gates (reset, update)
Handles long sequences	Struggles (vanishing gradients)	Yes	Yes
Parameters	Fewest	Most	Fewer than LSTM
Computational cost	Lowest	Highest	Lower than LSTM
Typical use cases	Simple/short sequences	Complex, long sequences	Good performance with less complexity

5 When to use a GRU?

- When you want the benefits of gating but with a simpler, more efficient model than LSTM.
- Works well for many NLP tasks (e.g. language modelling, text classification, named entity recognition).
- Often performs comparably to LSTM, especially on smaller datasets.
- Faster to train and uses fewer parameters, which can be useful when computational resources are limited.

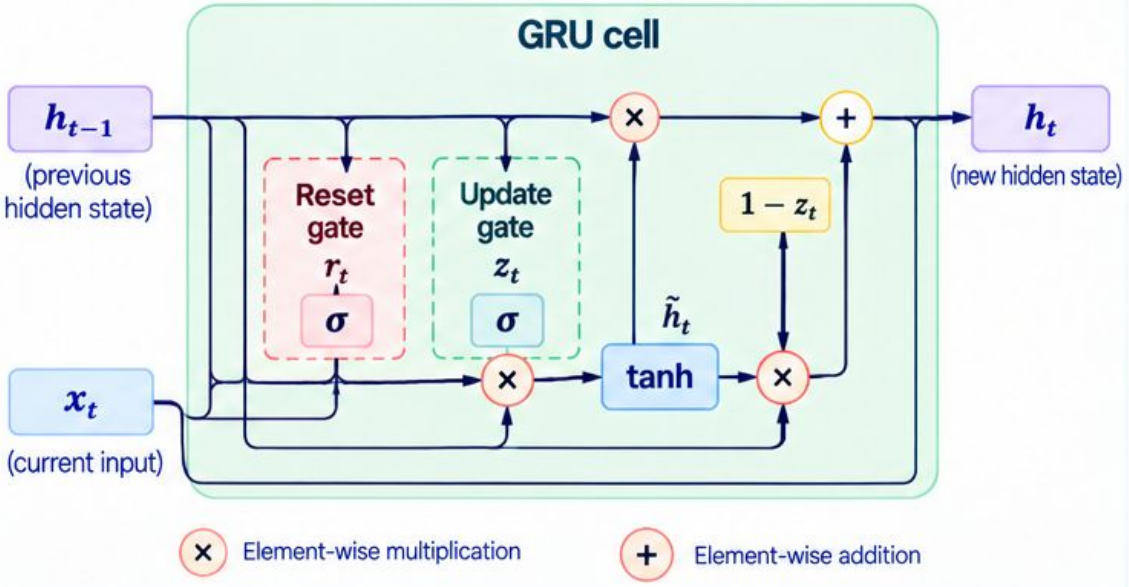
6 Key takeaway

A GRU is a simpler alternative to LSTM that uses two gates (reset and update) and a single hidden state, yet still effectively captures long-term dependencies in sequential data.

The GRU Cell — Basic Architecture

A GRU (Gated Recurrent Unit) uses two gates — a reset gate and an update gate — to control information flow. Unlike an LSTM, a GRU has a single hidden state and no separate cell state, making it simpler and more efficient.

1 GRU cell architecture



2 Reset gate (r_t)

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Controls how much of the past hidden state to forget when computing the candidate state.

- r_t close to 0 → ignore past information
- r_t close to 1 → keep past information

3 Update gate (z_t)

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Decides how much of the new candidate state to use versus the previous hidden state.

- z_t close to 0 → use more new information
- z_t close to 1 → keep more of the past state

4 Candidate hidden state ($\tilde{h}_t$)

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

Computes the new candidate hidden state using the reset gate to filter the past state.

5 Final hidden state (h_t)

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

A combination of the previous hidden state and the new candidate state, controlled by the update gate.

6 Key features of the GRU architecture

- Uses two gates: reset gate and update gate.
- Maintains a single hidden state (no separate cell state).
- Fewer parameters than an LSTM.
- Simpler architecture, often faster to train.
- Effectively captures long-term dependencies.
- Widely used in NLP tasks such as text classification, named entity recognition and machine translation.

7 Summary of equations

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r) \quad \text{(reset gate)}$$

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z) \quad \text{(update gate)}$$

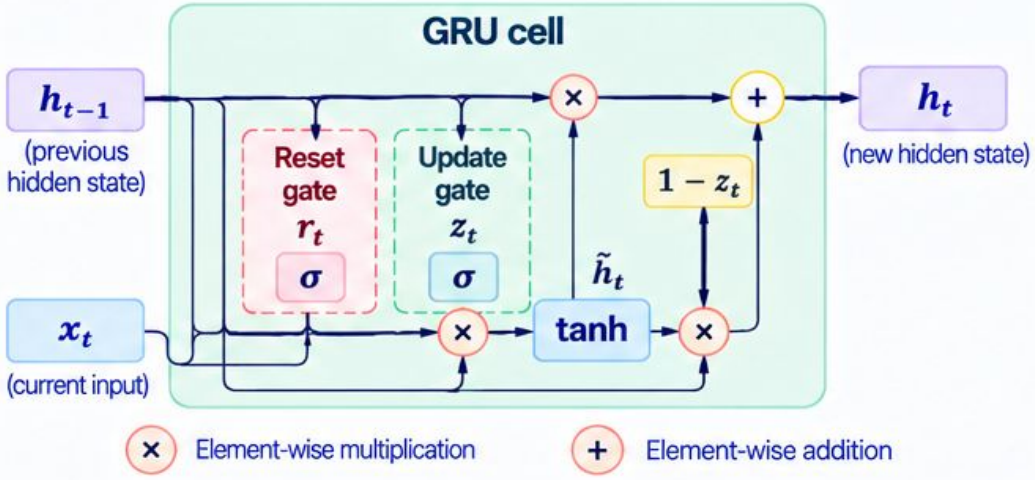
$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h) \quad \text{(candidate state)}$$

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1} \quad \text{(new hidden state)}$$

The Two GRU Gates — Reset & Update

A Gated Recurrent Unit (GRU) uses two gates to control the flow of information: a reset gate to decide how much past information to forget when creating new content, and an update gate to decide how much of the new content to keep.

1 GRU cell with gates



2 Reset gate (r_t)

$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$$

Controls how much of the past hidden state to forget when computing the candidate hidden state $\tilde{h}_t$.

Intuition

- r_t looks at the current input x_t and the previous hidden state h_{t-1}.
- r_t close to 0 → ignore most of the past hidden state (reset).
- r_t close to 1 → use most of the past hidden state.

3 Update gate (z_t)

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

Controls how much of the new candidate hidden state to keep versus the previous hidden state.

Intuition

- z_t looks at the current input x_t and the previous hidden state h_{t-1}.
- z_t close to 0 → use more of the new candidate state $\tilde{h}_t$ (update more).
- z_t close to 1 → keep more of the previous hidden state h_{t-1} (change less).

4 How the gates are used

1. Compute the reset gate r_t.
2. Use r_t to create the candidate hidden state $\tilde{h}_t$:

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$

3. Compute the update gate z_t.
4. Combine the candidate state and previous state:

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

5 Effect of gate values (examples)

Gate value	What it means	Effect
$r_t \approx 0$	Forget the past	Candidate state $\tilde{h}_t$ mainly based on current input x_t
$r_t \approx 1$	Keep the past	Candidate state $\tilde{h}_t$ uses more of h_{t-1}
$z_t \approx 0$	Update more	h_t relies more on the new candidate state $\tilde{h}_t$
$z_t \approx 1$	Keep previous	h_t stays close to h_{t-1} (small change)

6 Key takeaways

- A GRU uses two gates: a reset gate (r_t) and an update gate (z_t).
- The reset gate controls how much past information to forget when creating new content.
- The update gate controls how much of the new content to keep versus the previous hidden state.
- Together, these gates help the GRU capture long-term dependencies with fewer parameters than an LSTM.
- Widely used in NLP tasks such as text classification, named entity recognition and machine translation.

GRU — Worked NLP Example

Let's see how a GRU processes a sentence in a real NLP task. We use the example *"The movie was great"* and show how the GRU updates its hidden state at each word, and how the final hidden state is used for sentiment classification.

NLP task

Sentiment classification: Predict whether the sentence expresses a positive or negative sentiment.

Input: The movie was great

Output: Positive

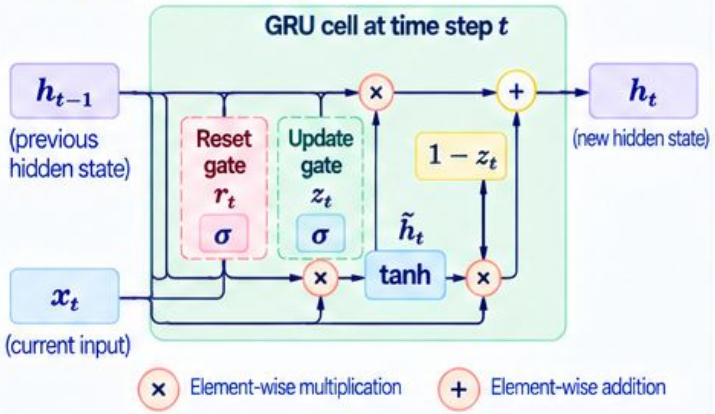
1 Input sentence and word embeddings

Each word is converted to a vector (embedding) that captures its meaning (e.g. using pre-trained embeddings such as GloVe or word2vec).

t	Word	Embedding (4-dimensional example)
1	The	[0.10, 0.05, -0.02, 0.01]
2	movie	[0.20, 0.10, -0.10, 0.05]
3	was	[0.15, 0.03, -0.05, -0.02]
4	great	[0.30, 0.25, 0.20, 0.10]

2 The GRU cell

A GRU uses two gates — a reset gate and an update gate — and a single hidden state h_t.



3 GRU equations

Reset gate (r_t)

$$r_t = \sigma(W_r [h_{t-1}, x_t] + b_r)$$

Controls how much of the past hidden state to forget.

Update gate (z_t)

$$z_t = \sigma(W_z [h_{t-1}, x_t] + b_z)$$

Controls how much of the new content to keep.

Candidate hidden state ($\tilde{h}_t$)

$$\tilde{h}_t = \tanh(W_h [r_t \odot h_{t-1}, x_t] + b_h)$$

New candidate hidden state using the reset gate.

Final hidden state (h_t)

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

Interpolation between old state and new candidate state.

σ = sigmoid function tanh = hyperbolic tangent $\odot$ = element-wise multiplication

4 Step-by-step example (conceptual values)

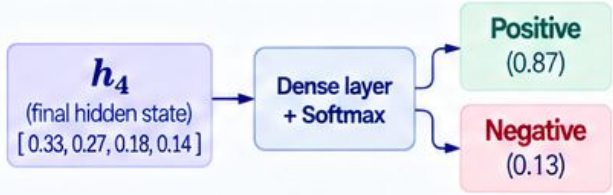
The GRU updates its hidden state at each word, keeping track of the meaning of the sentence.

t	Word	r_t (reset)	z_t (update)	$\tilde{h}_t$ (candidate)	h_t (hidden state)
0	–	–	–	–	[0.00, 0.00, 0.00, 0.00]
1	The	0.52	0.71	[0.12, 0.08, -0.04, 0.03]	[0.09, 0.06, -0.03, 0.02]
2	movie	0.48	0.63	[0.18, 0.14, -0.08, 0.05]	[0.14, 0.11, -0.06, 0.04]
3	was	0.55	0.67	[0.10, 0.06, -0.02, 0.04]	[0.12, 0.08, -0.04, 0.04]
4	great	0.37	0.28	[0.42, 0.35, 0.28, 0.18]	[0.33, 0.27, 0.18, 0.14]

Values are illustrative (not from a real model). They show how the GRU gradually builds a representation of the sentence, with the final hidden state reflecting the positive sentiment from the word "great".

5 Final prediction (sentiment classification)

The final hidden state h_T is passed to a dense layer with a softmax (or sigmoid) to predict the sentiment.



The model predicts that the sentence is positive, because the final hidden state captures the meaning of "great" in context.

6 Key takeaway

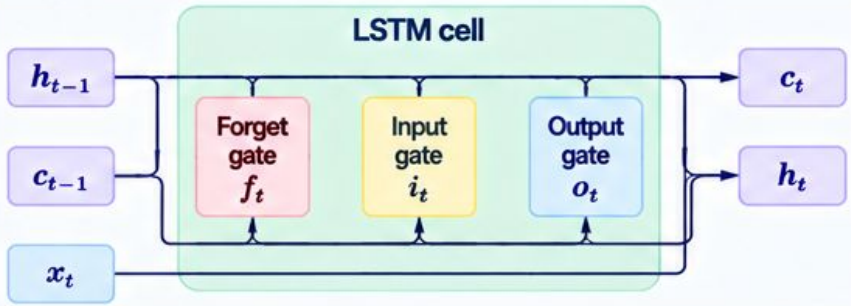
- A GRU uses two gates (reset and update) and a single hidden state.
- The reset gate helps the model ignore unimportant past information.
- The update gate controls how much new information to keep.
- GRUs are simpler than LSTMs, have fewer parameters, and work well for many NLP tasks such as text classification, named entity recognition and machine translation.
- The final hidden state can be used to make a prediction about the whole sentence.

LSTM vs GRU

Both LSTM and GRU are designed to address the vanishing gradient problem in RNNs and capture long-term dependencies. They are similar, but differ in their architecture, number of gates and complexity.

LSTM (Long Short-Term Memory)

Three gates and two states (hidden state and cell state).

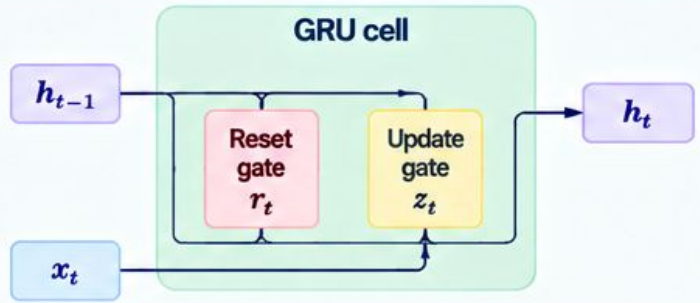


$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$

$$h_t = o_t \odot \tanh(c_t)$$

GRU (Gated Recurrent Unit)

Two gates and a single hidden state.



$$\tilde{h}_t = \tanh(W_h [r_t \odot h_{t-1}, x_t] + b_h)$$

$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

Key intuition

- LSTM uses a separate cell state c_t to explicitly store long-term information, controlled by three gates.
- GRU simplifies the design by combining the cell state and hidden state into a single hidden state h_t, with two gates.
- Both help the model remember useful information over long sequences and avoid the vanishing gradient problem.
- In practice, GRUs are simpler and often faster to train, while LSTMs can be more expressive and may perform better on very complex tasks.

Aspect	LSTM	GRU	Comments
Architecture	Three gates (forget, input, output)	Two gates (reset, update)	GRU is a simplified version of LSTM
States	Two states: cell state c_t and hidden state h_t	Single hidden state h_t	LSTM keeps a separate memory cell
Equations	More complex	Simpler	Fewer parameters in GRU
Parameters	More (larger model)	Fewer (smaller model)	GRU trains faster and uses less memory
Ability to capture long-term dependencies	Often better on very long and complex sequences	Good performance, slightly less expressive	Both handle long-term dependencies well
Training time	Slower	Faster	GRU is more efficient
Typical use cases	Complex sequence tasks (e.g. machine translation, speech recognition)	Many NLP tasks (e.g. text classification, named entity recognition, sentiment analysis)	GRU is often a good default choice

Takeaway

Use LSTM when you need maximum modelling capacity and are dealing with very long or complex sequences.
Use GRU when you prefer a simpler, faster model that performs well on most NLP tasks.

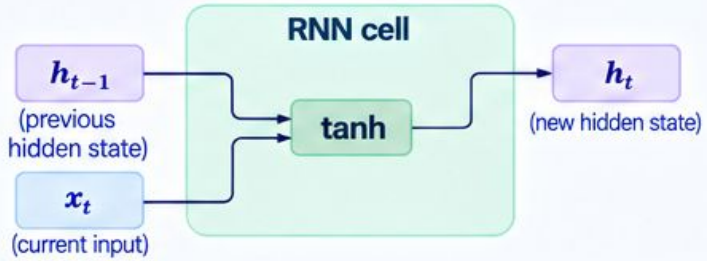
RNN vs LSTM vs GRU

"Same idea, different memory."
RNN → LSTM → GRU

Three recurrent neural network architectures for sequence modelling — from simple to more advanced memory mechanisms.

1 Vanilla RNN

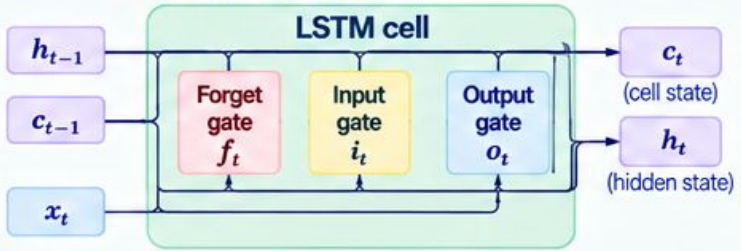
A simple recurrent network with a single hidden state.



$$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$$

2 LSTM (Long Short-Term Memory)

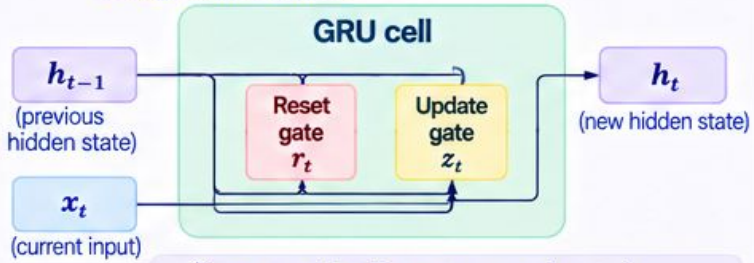
Uses three gates and a separate cell state to control information flow.



$$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$$
$$h_t = o_t \odot \tanh(c_t)$$

3 GRU (Gated Recurrent Unit)

A simpler variant of LSTM with two gates and a single hidden state.



$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$$
$$h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$$

Aspect	RNN	LSTM	GRU
Architecture	Single recurrent layer (tanh)	Three gates: forget, input, output	Two gates: reset, update
Memory mechanism	Single hidden state h_t	Hidden state h_t and separate cell state c_t	Single hidden state h_t
Gates	None	3 gates (f_t, i_t, o_t)	2 gates (r_t, z_t)
Equations	$h_t = \tanh(W_h h_{t-1} + W_x x_t + b)$	$c_t = f_t \odot c_{t-1} + i_t \odot \tilde{c}_t$ $h_t = o_t \odot \tanh(c_t)$	$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h)$ $h_t = (1 - z_t) \odot \tilde{h}_t + z_t \odot h_{t-1}$
Number of states	1 (hidden state)	2 (hidden state + cell state)	1 (hidden state)
Number of parameters	Lowest	Highest	Fewer than LSTM (typically between RNN and LSTM)
Ability to capture long-term dependencies	Poor (vanishing gradients)	Very good	Good
Training complexity	Easiest	More complex	Simpler than LSTM
Computational cost	Lowest	Highest	Lower than LSTM
Typical use cases (NLP)	Simple/short sequences, educational use	Complex sequences (e.g. machine translation, speech recognition, document modelling)	Many NLP tasks (e.g. text classification, named entity recognition, sentiment analysis)

Key takeaway: All three models are useful for sequence modelling. Start with a simple RNN to understand the idea, use a GRU when you want a simpler and faster model, and use an LSTM when you need maximum memory capacity for very long or complex sequences.

Different designs, same goal: learn from sequences.

Applications of RNN, LSTM & GRU in NLP

RNN, LSTM and GRU are widely used in NLP to model sequential text data. They help capture context, word order and long-term dependencies, and are effective for many real-world language tasks.

Same idea. Different architectures. Real-world impact.

1 Language Modelling

Predict the next word in a sequence.

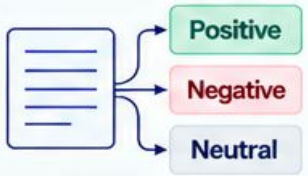


Example:
Given "The movie was", predict "great".

Common architectures:
RNN | LSTM | GRU

2 Text Classification

Assign a label to a text sequence (e.g. sentiment, topic, spam).



Example:
Classify the sentiment of "The movie was great!".

Common architectures:
RNN | LSTM | GRU

3 Named Entity Recognition

Identify named entities in text (e.g. person, location, organisation).

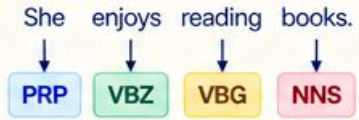


Example:
Tag "Barack Obama visited Mauritius."

Common architectures:
RNN | LSTM | GRU

4 Part-of-Speech Tagging

Assign a grammatical tag to each word.



Example:
Tag each word in "She enjoys reading books."

Common architectures:
RNN | LSTM | GRU

5 Machine Translation

Translate text from one language to another.

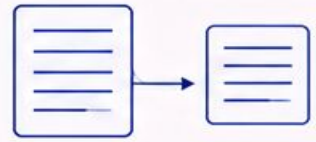


Example:
French → English translation using sequence-to-sequence.

Common architectures:
LSTM | GRU

6 Text Summarisation

Generate a shorter version of a longer text.



Example:
Summarise a news article into a few sentences.

Common architectures:
LSTM | GRU

Where each model is commonly used

Task / Application	RNN	LSTM	GRU
Language modelling	✓	✓	✓
Text classification	✓	✓	✓
Named entity recognition	✓	✓	✓
Part-of-speech tagging	✓	✓	✓
Machine translation	–	✓	✓
Text summarisation	–	✓	✓
Dialogue systems / chatbots	–	✓	✓
Question answering	–	✓	✓
Other sequence tasks	✓	✓	✓

Typical use case examples

Task	Example use case
Language modelling	Predict next word (e.g. for autocomplete)
Text classification	Sentiment analysis (positive/negative)
Named entity recognition	Extract names, dates, locations
Part-of-speech tagging	Identify nouns, verbs, adjectives
Machine translation	Translate English ↔ French
Text summarisation	Summarise news, legal documents
Dialogue systems	Build chatbots or virtual assistants
Question answering	Answer questions from a passage
Speech-to-text (with RNN variants)	Convert speech to text (using recurrent or hybrid models)

Why LSTM and GRU are popular in NLP

- Address the vanishing gradient problem, allowing them to capture long-term dependencies.
- Work well with variable-length sequences (e.g. sentences of different lengths).
- Achieve strong performance on many NLP tasks.
- GRUs are simpler and faster, often giving comparable results to LSTMs.
- LSTMs can be more expressive and may perform better on very long or complex sequences.

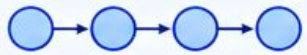
Key takeaway

- All three architectures can be used in NLP.
- Choose RNN for simple or educational purposes.
- Choose GRU for a simpler, faster model with good performance.
- Choose LSTM when you need to capture very long or complex dependencies.
- The best choice depends on your task, data and resource constraints.

Limitations of RNN, LSTM and GRU

While RNN, LSTM and GRU have been very useful in NLP, they also have important limitations. Understanding these helps us choose the right model and motivates newer architectures (e.g. Transformers).

RNN
(Vanilla Recurrent Neural Network)



Simple, but struggles with long-term dependencies.

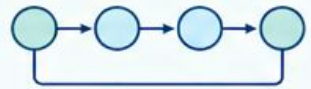
Key limitations

- **Vanishing and exploding gradients:** Difficult to learn long-term dependencies.
- **Limited memory:** Single hidden state, no gating mechanism to control information flow.
- **Poor performance on long sequences:** Forgets information from earlier time steps.
- **Sensitive to weight initialisation and training** setup.
- **Less effective than LSTM/GRU** for complex NLP tasks.

Example in NLP

May perform well on short text (e.g. short sentence classification) but struggles with long documents or dialogue history.

LSTM
(Long Short-Term Memory)



Addresses vanishing gradient problem, but comes with higher complexity.

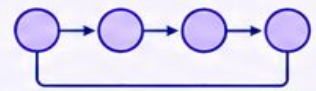
Key limitations

- **More complex architecture:** Three gates (forget, input, output) and a separate cell state.
- **Higher computational cost:** More parameters and slower training.
- **Overfitting risk:** Can overfit on small datasets.
- **Not always significantly better than GRU:** In many practical tasks, the performance gain over GRU is small.
- **Still struggles with very long sequences:** Although better than vanilla RNN, it may still lose some long-range information.

Example in NLP

Works well for tasks like machine translation or document modelling, but training can be slow and resource intensive, especially with very long texts.

GRU
(Gated Recurrent Unit)



Simpler than LSTM, but with some trade-offs.

Key limitations

- **Fewer gates:** Only reset and update gates, which makes it less expressive than LSTM in some cases.
- **May capture less complex patterns:** Can be outperformed by LSTM on very long or complex sequences.
- **Still affected by vanishing gradients:** Although less severe than vanilla RNN, it can still struggle with very long dependencies.
- **Less control over memory:** Does not have a separate cell state, which can limit its ability to retain information in some tasks.

Example in NLP

Performs well on many NLP tasks (e.g. text classification, named entity recognition) and is often faster to train, but may be less effective than LSTM for highly complex sequences.

Summary comparison

Limitation	RNN	LSTM	GRU
Vanishing gradients	Severe	Reduced	Reduced
Model complexity	Low	High	Medium
Training time	Fastest	Slowest	Faster than LSTM
Ability to handle long sequences	Poor	Good	Good (but sometimes less than LSTM)
Risk of overfitting	Lower	Higher	Lower
Expressiveness	Low	High	Medium

Key takeaways

1. RNN is simple but struggles with long-term dependencies.
2. LSTM overcomes many of RNN's limitations but is more complex and computationally expensive.
3. GRU offers a good balance between performance and efficiency, but may be less expressive than LSTM.
4. All three can still face challenges with very long sequences and large-scale NLP tasks.
5. For many modern NLP applications, Transformer models have become the preferred choice, as they overcome most sit these limitations using self-attention.

UNIVERSITY OF MAURITIUS
Knowledge for a Better World

SIS 3204Y(5)
NATURAL LANGUAGE PROCESSING

WEEK 5
Recurrent Neural Networks for NLP
RNN, LSTM and GRU

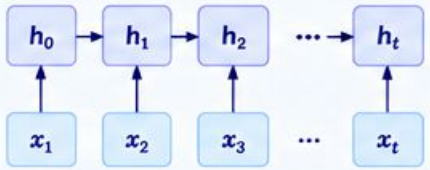
From Words to Meaning

From Recurrent Networks to Attention & Transformers

RNNs, LSTM and GRU process text sequentially. Attention overcomes their limitations by looking at all words in parallel. Transformers build on attention and have become the state-of-the-art in NLP.

Same goal:
Better language understanding.
Different ideas.
Bigger possibilities.

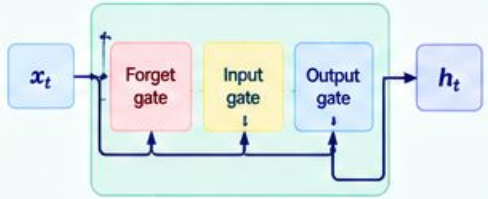
Vanilla RNN
The starting point



- Processes text step by step.
- Single hidden state.
- Simple architecture.
- Struggles with long-term dependencies (vanishing/exploding gradients).

1980s – 1990s
Early recurrent models

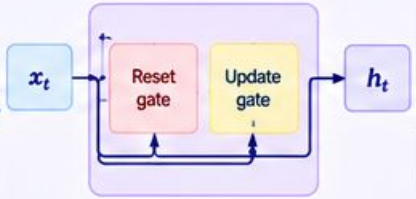
LSTM
Better memory for long sequences



- Introduces a memory cell and three gates (forget, input, output).
- Captures long-term dependencies.
- More parameters and computational cost than vanilla RNN.
- Widely used in NLP (e.g. language modelling, machine translation).

1997
LSTM (Hochreiter & Schmidhuber)

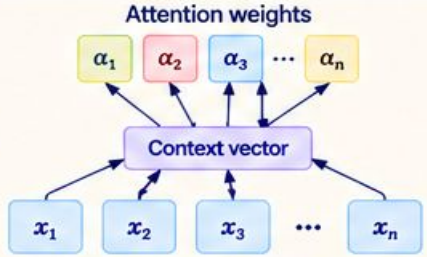
GRU
Simpler and more efficient



- Combines cell and hidden state.
- Only two gates (reset, update).
- Fewer parameters than LSTM.
- Often trains faster with similar performance.
- Popular for many NLP tasks.

2014
GRU (Cho et al.)

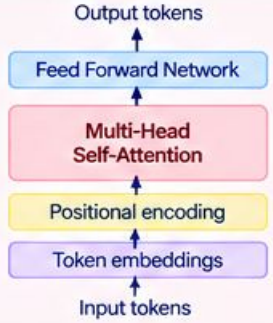
Attention Mechanism
Look at all words, not just the next one



- Computes relevance (attention weights) between all words in a sequence.
- Allows direct access to any position, regardless of distance.
- Alleviates the long-term dependency problem.
- Used in sequence-to-sequence models and forms the foundation of Transformers.

2014
Attention (Bahdanau et al.)

Transformer
Built on attention, no recurrence



- Uses self-attention (no recurrence).
- Processes all tokens in a parallel.
- Captures global context effectively.
- More scalable and efficient for large datasets.
- State-of-the-art performance in most NLP tasks (e.g. BERT, GPT, T5, LLaMA).

2017
Transformer (Vaswani et al.)

Key differences

Aspect	RNN	LSTM	GRU	Attention	Transformer
Processing	Sequential	Sequential	Sequential	Parallel (over sequence)	Parallel
Memory mechanism	Hidden state	Cell state + gates	Gates (no cell)	Attention weights	Self-attention
Handles long-term dependencies	Poor	Very good	Good	Very good	Very good
Training speed	Fastest	Slower	Faster	Fast	Fast (highly parallel)
Computational cost	Lowest	Highest	Lower than LSTM	Moderate	Scales well
Typical use in NLP	Simple tasks	Language modelling, translation (older systems)	Many NLP tasks	Seq2seq, NMT (early)	Modern NLP (BERT, GPT, etc.)

Why the shift to attention and Transformers?

- RNNs, LSTM and GRU process tokens sequentially, which limits parallelisation and makes long sequences hard to learn.
- Attention allows the model to focus on relevant words, regardless of their position.
- Transformers are more scalable, efficient and achieve better performance on large datasets.
- They can capture complex, long-range dependencies and rich contextual information.
- This makes them especially effective for modern NLP tasks such as machine translation, text summarisation, question answering and large language models (e.g. BERT, GPT, LLaMA).

Key takeaway

RNN → LSTM → GRU → Attention → Transformer

Each step addresses the limitations of the previous one, leading to more powerful models for understanding and generating human language.

Learn from the past,
Build for the future.

UNIVERSITY OF MAURITIUS
Knowledge for a Better World

SIS 3204Y(5)
NATURAL LANGUAGE PROCESSING

WEEK 5
Recurrent Neural Networks for NLP
RNN, LSTM and GRU

From Words to Meaning

Week 5 — Final Recap

We explored recurrent neural networks (RNN, LSTM, GRU) for processing sequential text data, their applications and limitations, and how attention and Transformers overcome these limitations to achieve state-of-the-art performance in NLP.

Same concepts.
Bigger perspective.
You now have the tools to choose, apply and go further!

1 RNN — The Starting Point

Simple recurrent network



- Single hidden state.
- No gates.
- Simple architecture, few parameters.
- Works for short sequences.
- Struggles with long-term dependencies (vanishing/exploding gradients).

2 LSTM — Better Memory

Uses gates to control information flow



- Two states: hidden state and cell state.
- Three gates (forget, input, output).
- Captures long-term dependencies.
- More parameters and higher computational cost.
- Performs well on complex sequence problems.

3 GRU — Simpler and Efficient

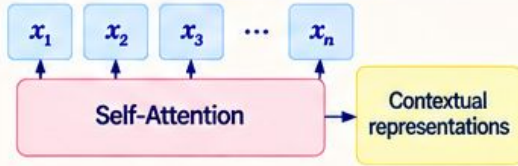
A simplified variant of LSTM



- Single hidden state.
- Two gates (reset, update).
- Fewer parameters than LSTM.
- Often trains faster with similar performance.
- Widely used in NLP tasks.

4 From RNNs to Attention & Transformers

Attention looks at all words in parallel



- No recurrence — processes all tokens in parallel.
- Learns attention weights to focus on relevant words.
- More scalable and efficient for long sequences.
- State-of-the-art performance in most NLP tasks.
- Examples: BERT, GPT, T5, LLaMA.

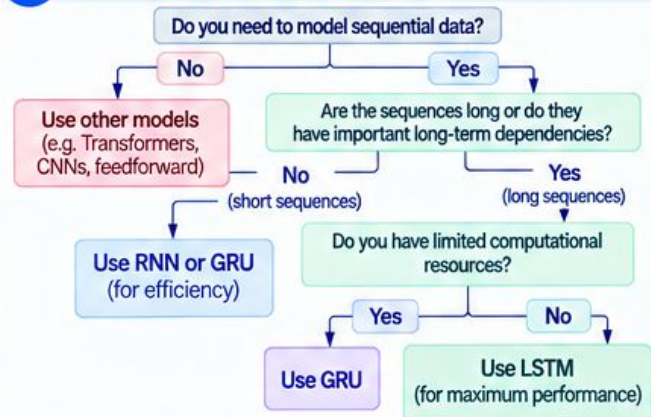
5 Applications in NLP

- Language modelling (next word prediction)
- Text classification (sentiment, topic, spam)
- Named entity recognition (person, location, organisation)
- Part-of-speech tagging
- Machine translation
- Text summarisation
- Dialogue systems / chatbots
- Question answering

6 Key Limitations

Limitation	RNN	LSTM	GRU
Vanishing / exploding gradients	Severe	Reduced	Reduced
Memory mechanism	Single state	Cell state + gates	Gates (no cell)
Model complexity	Low	High	Medium
Training time	Fastest	Slower	Faster
Handles long sequences	Poor	Good	Good (but sometimes < LSTM)
Computational cost	Lowest	Highest	Lower than LSTM
Expressiveness	Low	High	Medium

7 Choosing the Right Model



8 Key Takeaways

- RNN, LSTM and GRU process text sequentially using a hidden state.
- LSTM and GRU address the vanishing gradient problem, allowing long-term dependencies.
- GRU is simpler and often as effective as LSTM for many NLP tasks.
- Attention and Transformers overcome recurrence limitations by processing all words in parallel.
- Choose the model based on your task, data size, computational resources and required performance.

Different tools. Real-world impact.

9 What's Next?

- Implement RNN, LSTM and GRU on a real NLP task (e.g. text classification or language modelling).
- Compare their performance on the same dataset.
- Explore Attention and Transformers in the next weeks (e.g. BERT).
- Think about how these models can be applied to real problems in your domain of interest.

"The models are tools. Your ideas give them meaning."